

How Low Can We Go? Minimizing Interaction Samples for Configurable Systems

DOMINIK M. KRUPKE, AHMAD MORADI, MICHAEL PERK, and PHILLIP KELDENICH,
Algorithms Division, TU Braunschweig, Braunschweig, Germany
GABRIEL GEHRKE, TU Braunschweig, Braunschweig, Germany
SEBASTIAN KRIETER and THOMAS THÜM, Institute of Software Engineering and Automotive
Informatics (ISF), TU Braunschweig, Braunschweig, Germany
SÁNDOR P. FEKETE, Algorithms Division, TU Braunschweig, Braunschweig, Germany

Modern software systems are typically configurable, a fundamental prerequisite for wide applicability and reusability. This flexibility poses an extraordinary challenge for quality assurance, as the enormous number of possible configurations makes it impractical to test each of them separately. This is where *t-wise interaction sampling* can be used to systematically cover the configuration space and detect unknown feature interactions. Over the last two decades, numerous algorithms for computing small interaction samples have been studied, providing improvements for a range of heuristic results; nevertheless, it has remained unclear how much these results can still be improved.

We present a significant breakthrough: a fundamental framework, based on the mathematical principle of *duality*, for combining near-optimal solutions with provable lower bounds on the required sample size. This implies that we no longer need to work on heuristics with marginal or no improvement, but can certify the solution quality by establishing a limit on the remaining gap; in many cases, we can even prove optimality of achieved solutions. This theoretical contribution also provides extensive practical improvements: Our algorithm SampLNS was tested on 47 small- and medium-sized configurable systems from the existing literature. SampLNS can reliably find samples of smaller size than previous methods in 85% of the cases; moreover, we can achieve and prove optimality of solutions for 63% of all instances. This makes it possible to avoid cumbersome efforts of minimizing samples by researchers as well as practitioners, and substantially save testing resources for most configurable systems.

All authors contributed significantly to the work. The order of the first three authors is alphabetical.

This work was partially supported through the projects “Quantum Methods and Benchmarks for Resource Allocation” (QuBRA), German Federal Ministry for Education and Science (BMBF), grant number 13N16053 and “Quantum Readiness for Optimization Providers” (ProvideQ), and German Federal Ministry of Economic Affairs and Climate Action (BMWK), grant number 01MQ22006A.

Authors’ Contact Information: Dominik M. Krupke (corresponding author), Algorithms Division, TU Braunschweig, Braunschweig, Germany; e-mail: krupke@ibr.cs.tu-bs.de; Ahmad Moradi, Algorithms Division, TU Braunschweig, Braunschweig, Germany; e-mail: moradi@ibr.cs.tu-bs.de; Michael Perk, Algorithms Division, TU Braunschweig, Braunschweig, Germany; e-mail: perk@ibr.cs.tu-bs.de; Phillip Keldenich, Algorithms Division, TU Braunschweig, Braunschweig, Germany; e-mail: keldenich@ibr.cs.tu-bs.de; Gabriel Gehrke, TU Braunschweig, Braunschweig, Germany; e-mail: ggehrke@ibr.cs.tu-bs.de; Sebastian Krieter, Institute of Software Engineering and Automotive Informatics (ISF), TU Braunschweig, Braunschweig, Germany; e-mail: sebastian.krieter@tu-braunschweig.de; Thomas Thüm, Institute of Software Engineering and Automotive Informatics (ISF), TU Braunschweig, Braunschweig, Germany; e-mail: thomas.thuem@tu-braunschweig.de; Sándor P. Fekete, Algorithms Division, TU Braunschweig, Braunschweig, Germany; e-mail: s.fekete@tu-bs.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/7-ART175

<https://doi.org/10.1145/3712193>

CCS Concepts: • **Software and its engineering** → **Software product lines; Software product lines;**
 • **Mathematics of computing** → **Combinatorial optimization; Combinatorial optimization;**

Additional Key Words and Phrases: Configurable System, Software Product Lines, T-Wise Sampling, Product-Based Testing, Large Neighborhood Search, Mixed Integer Programming, Constraint Programming, Algorithm Engineering

ACM Reference format:

Dominik M. Krupke, Ahmad Moradi, Michael Perk, Phillip Keldenich, Gabriel Gehrke, Sebastian Krieter, Thomas Thüm, and Sándor P. Fekete. 2025. How Low Can We Go? Minimizing Interaction Samples for Configurable Systems. *ACM Trans. Softw. Eng. Methodol.* 34, 6, Article 175 (July 2025), 29 pages.

<https://doi.org/10.1145/3712193>

1 Introduction

Modern software systems often provide hundreds or thousands of configuration options (i.e., *features*) that enable customization of these systems to specific user requirements [42, 82, 88]. A challenge for testing these highly configurable systems are potential interactions between features, which can lead to unforeseen behavior and faults in configurations containing a particular feature combination [1, 10, 17, 18]. Due to the enormous number of potential feature interactions in configurable systems, detecting such faults is challenging [1]. A popular solution for finding faults in a configurable system is product-based testing in combination with sampling methods to create a representative list of configurations (i.e., a *sample*), which are then tested individually [19, 92, 93]. The number of configurations, known as the sample size, is a common proxy for approximating the testing effort. Thus, minimizing the sample size can reduce the overall testing effort [35]. However, to ensure reliability, it is typically necessary to test each interaction between up to t features [28, 72]. This yields the **T-WISE INTERACTION SAMPLING PROBLEM (t -ISP)**: find a sample consisting of a minimum number of configurations, such that every valid combination of t or less features is part of at least one configuration, where t is an arbitrary but fixed positive integer.

In addition to its practical importance for modern software systems, Interaction Sampling is also highly challenging from a theoretical point of view, as it contains the classical problem SET COVER as a special case: Given a universe $\mathcal{U}$ of elements $\{u_1, \dots, u_n\}$ and a family $\mathcal{S} = \{S_1, \dots, S_m\}$ of subsets of $\mathcal{U}$, each with a nonnegative cost $c(S_j)$, find a subfamily $\mathcal{C} := \{S_j \mid j \in J\}$ with $J \subset \{1, \dots, m\}$, such that $\bigcup_{j \in J} S_j = \mathcal{U}$ and $\sum_{j \in J} c(S_j)$ is minimal. SET COVER is an NP-hard problem [47], so we cannot expect to develop a deterministic algorithm that can guarantee to compute a provably optimal solution in a worst-case runtime whose asymptotic growth is bounded by a polynomial in the input size. This also implies that not only may it be difficult to *find* a good solution, it is also hard to *certify* that an identified solution is indeed optimal, because the logical complement (prove that there is *no* solution that is better) is coNP-hard.

This combination of practical importance with theoretical difficulty of t -ISP makes it plausible to resort to mere heuristics, such as greedy or evolutionary approaches, which aim at finding decent solutions by local improvement. (Even though some algorithms may be able to generate optimal samples [97] for small instances by solving a large incremental SAT formulation, their usefulness is limited by their lack of scalability.) While such heuristics may yield decent samples [92], they cannot be expected to reliably compute optimal solutions; what is more, they do not provide any estimate on solution quality, so there is no way of knowing how much potential for further improvement exists—or even to certify whether an obtained solution happens to be optimal. As a consequence, the sample size achieved by a new algorithm for t -ISP can only be compared to other existing algorithms [68]. However, this means the achieved sample size is compared only

against an *upper bound* (i.e., the smallest known sample size for which there exists a sample that achieves full t -wise coverage), but not to a lower bound or even an optimal sample. Currently, there exist dozens of different t -wise sampling algorithms [3, 64, 92], for which the absolute performance is unknown, effectively using a lower bound of zero. In the absence of better lower bounds, it is completely unclear whether investigating performance improvements of these algorithms is a promising or a futile endeavor.

This is where Mathematical Optimization and Algorithm Engineering offer mathematically rigorous and practically useful methods. Over the years, these areas have developed more refined ways to deal with the practical difficulties of finding good solutions for relevant instances of hard optimization problems. One success story is offered by the well-known TRAVELING SALESMAN PROBLEM of finding a shortest roundtrip through a given number of locations, where such methods have made it possible to compute *provably optimal* solutions to instances with as many as 13,509 cities [11], which amounts to both *finding* a best of $13,509! \approx 10^{50,000}$ possible roundtrips and also *proving* that none of the others is better; see [25] for full details.

A crucial concept in this endeavor is the notion of *duality*. For a given “primal” minimization problem $\mathcal{A}$ with a cost function a , the idea is to identify a suitable “dual” maximization problem $\mathcal{B}$ with a cost function b , such that for any (not necessarily optimal) feasible solution x of $\mathcal{A}$ and any feasible solution y of $\mathcal{B}$, the objective values $a(x)$ of x and $b(y)$ of y satisfy $a(x) \geq b(y)$. Thus, once a suitable problem $\mathcal{B}$ has been identified and duality has been established, *any* feasible solution y of $\mathcal{B}$ provides a lower bound for the optimal value of $\mathcal{A}$, independent of how it was obtained. This key observation allows it to combine a variety of approaches (including local search for feasible solutions of $\mathcal{A}$ and $\mathcal{B}$) with mathematically rigorous quality guarantees of the obtained feasible solutions. In particular, if we manage to find feasible solutions x^* for $\mathcal{A}$ and y^* for $\mathcal{B}$, such that $a(x^*) = b(y^*)$, then any feasible solution x of $\mathcal{A}$ must satisfy $a(x) \geq b(y^*) = a(x^*)$, so y^* provides a certificate for the optimality of x^* for $\mathcal{A}$; this holds regardless of whether x^* and y^* were obtained by a time-consuming, systematic search or by a lucky guess. Even if establishing optimality by matching dual solutions turns out to be time consuming or elusive, we may be able to quickly establish a limited gap to an optimal solution; this is particularly useful for difficult optimization problems for which finding *some* good solution is usually achieved much faster than proving that *no* better solution exists. (See Section 2.4 for a slightly more detailed technical introduction.)

Our Contributions. We present a powerful algorithm for computing upper and lower bounds for the problem of minimizing the sample size in software configuration problems such as the t -ISP, along with a mathematically rigorous framework for providing performance metrics. While achieving tight lower or upper bounds (and thus, optimality) cannot be guaranteed in general, the practical applicability of our method is underscored through the provision of optimal or nearly optimal samples across a broad range of benchmark instances, accompanied by quality certificates. Notably, we were able to obtain a provably minimal sample size for the feature model EMBToolkit with 1,179 features and 5,414 clauses. In detail, we provide the following:

- We present a strategy for computing provable lower bounds for the optimum sample size in the t -ISP, based on establishing a suitable problem and a mathematical proof of duality.
- We introduce an algorithm that minimizes the sample size with full-coverage guarantees and provides a quality certificate, i.e., lower bound.
- We demonstrate the practical usefulness of our approach by a comparison of 47 small to medium-sized configurable systems. The outcomes are substantial improvements (on 60% of the instances we have an improvements of at least 20%), and tight performance bounds for 55% of the instances.
- We provide all source code and data for reproduction and verification of our results.

2 Preliminaries

In this section, we formally define feature models and the t -ISP. We also introduce key algorithmic techniques that will be employed later in our approach, including **Mixed Integer Programming (MIP)** and **Constraint Programming (CP)**. While these techniques can have exponential runtimes, making them impractical for certain problem sizes, they remain powerful tools for solving many otherwise intractable problems. Additionally, we present **Large Neighborhood Search (LNS)** as a practical heuristic for scaling these techniques, and discuss the concepts of duality and certificates, which can be used to establish quality guarantees even when applying heuristic methods like LNS.

2.1 Validity of Configurations

The dependencies in a reconfigurable system can be expressed by defining the set of valid configurations as those that satisfy a Boolean formula. To this end, a *feature model* $M = (\mathcal{F}, \mathcal{D})$ consists of n features $\mathcal{F} = \{1, 2, \dots, n\}$ and m dependencies $\mathcal{D} = \{\mathcal{D}_1, \mathcal{D}_2, \dots, \mathcal{D}_m\}$. We denote the literals of M by $\mathcal{L} = \{-n, \dots, -1, 1, \dots, n\}$, where $i \in \mathcal{L}$ represents the *activated* feature $|i|$, if $i > 0$ and the *deactivated* feature otherwise. A configuration can be expressed as a set of *literals* $C \subseteq \mathcal{L}$, such that for every feature $i \in \mathcal{F}$ either $i \in C$ or $-i \in C$. For simplicity, let the dependencies $\mathcal{D}$ be given in conjunctive normal form, for which each clause $\mathcal{D}_i \in \mathcal{D}$ is a subset of literals $\mathcal{L}$. Every Boolean formula can be expressed in this form, possibly with the help of auxiliary features [58]. We denote the set of all valid configurations by $||M||$. The validity of a configuration C is determined by $\forall \mathcal{D}_i \in \mathcal{D} : \mathcal{D}_i \cap C \neq \emptyset$. Thus, a valid configuration has to contain a literal of each clause $\mathcal{D}_i \in \mathcal{D}$. Note that the definition based on the conjunctive normal form is purely for readability; our actual implementation supports the direct use of more advanced constraints.

2.2 t -Wise Interaction Samples

In t -wise interaction sampling, an *interaction* I is represented by a subset $I \subseteq \mathcal{L}$ of t literals; it is considered *valid* iff there exists a valid configuration $C \in ||M||$ with $I \subseteq C$. Correspondingly, an interaction I is considered *invalid*, if no valid configuration C with $I \subseteq C$ exists and thus, it can never appear in any instantiation of the system. For a set of valid interactions $\mathcal{I}$, a *sample* $S = \{C_1, C_2, \dots\} \subseteq ||M||$ is a set of valid configurations such that $\forall I \in \mathcal{I} : \exists C_i \in S : I \subseteq C_i$, i.e., every interaction of $\mathcal{I}$ is contained in at least one of the configurations. We especially consider *complete pairwise interaction sampling*, for which $t = 2$ and $\mathcal{I}$ is the set of all valid interactions on the concrete features of $\mathcal{F}$, i.e., the features that are not added as composite or auxiliary features [91]. The underlying optimization problem t -ISP is to find a sample S for $\mathcal{I}$ with a minimum number of configurations.

2.3 Algorithmic Techniques

We employ a number of powerful algorithmic techniques and implementations. Fundamentally important are *SAT solvers* for finding feasible assignments or proving unsatisfiability for propositional formulas in conjunctive normal form. SAT solvers are typically based on an algorithm called Conflict-Driven Clause Learning [67], which learns new clauses that must also be satisfied whenever all clauses of the original formula are. Modern implementations, such as Kissat [16], can routinely solve instances with hundreds of thousands of variables in acceptable time. Such a solver is also utilized by other sampling algorithms, such as ICPL [44] or YASA [57], and is implicitly used by us via *CP-SAT*.

Also relevant are CP solvers, such as CP-SAT (a part of Google's ortools [75]), which facilitate not only considering more complex constraints and variables than a SAT solver, but also optimizing an objective function; this enables us to find not only feasible but actually optimal assignments.

By using a portfolio of techniques, especially *lazy clause generation* [29], CP-SAT can solve many optimization models with hundreds of thousands of variables. We show in Section 4.1 how to use this tool to compute samples of minimum size for small feature models.

A further highly effective technique to compute optimal solutions for combinatorial optimization problems is MIP. For a given set of linear constraints, such as $\sum_{j=1}^n a_{ij}x_j \leq b_i$, the goal is to minimize or maximize a linear objective function $\sum_{i=1}^n c_i x_i$ over a set of real or integer variables x_i . MIP is a well-known NP-hard problem, but commercial MIP solvers (such as IBM ILOG CPLEX or Gurobi [32]) can routinely solve instances of small to medium size in acceptable time to (near-) optimality. These solvers are based on the branch-and-bound algorithm template, refined with a large portfolio of techniques, including the generation of additional, valid *cut* constraints and sophisticated presolving.

Although neither CP-SAT nor MIP solvers scale effectively for our use case, we can still employ them for finding good upper and lower bounds by making them part of a LNS [80], a local optimization heuristic in which we repeatedly aim to replace a part of the current solution via an optimization problem. This is also called *destroy* and *repair*, as we essentially destroy a part of a solution by erasing some variable assignments, and then repair it via an optimization that tries to find the best values for the freed variables. For example, for our problem the destroy operation will delete a number of configurations from the solution, and the repair operation will try to find a minimal set of configurations to cover the now-missing interactions again. We can scale the extent of this destruction and the size of the resulting optimization problem for repair so that it can be solved to (near-)optimality using CP-SAT or MIP. The advantage of this approach over others (e.g., Genetic Algorithms or Simulated Annealing) is that we do not sequentially go through a pool of solutions, but can search in an exponentially sized space of improvements for the best solution, as they are encoded implicitly by the repair problem. There are generic implementations of LNS, such as Relaxation Induced Neighborhood Search [26], that have become important parts of modern MIP solvers; using domain-knowledge, one can engineer even more powerful implementations, as we demonstrate in this article.

2.4 Duality and Quality Certificates

As described in the Introduction, an essential concept utilized in this article is *duality*.

Definition 2.1. Let $\mathcal{A} : \min\{a(x) \mid x \in X\}$ be a (primal) minimization problem with feasible set X and cost function a . Then a maximization problem $\mathcal{B} : \max\{b(y) \mid y \in Y\}$ with feasible set Y and cost function b is *dual* to $\mathcal{A}$, if for any $x \in X$ and any $y \in Y$, the objective values $a(x)$ of x and $b(y)$ of y satisfy $a(x) \geq b(y)$.

For a classical example of duality, consider the problem MINIMUM VERTEX COVER, a special case of SET COVER, which aims at finding a smallest subset of vertices $S \subseteq V$ in a graph $G = (V, E)$, such that any edge $e \in E$ contains one of the vertices in S , and the problem MAXIMUM MATCHING, which asks for a maximum set $M \subseteq E$ of disjoint edges. If S is *any* feasible vertex cover and M is *any* feasible matching, then no two edges in M can contain the same vertex in S , so S must contain at least one separate vertex for each edge in M , and $|S| \geq |M|$.

By definition, any feasible solution for the dual provides a lower bound for the primal (i.e., the original) problem, and thus a quality certificate. Moreover, a suitable solution to the dual can provide a short certificate of optimality for a suitable solution to the primal problem.

LEMMA 2.2. *Consider a primal problem $\mathcal{A} : \min\{a(x) \mid x \in X\}$ and a dual problem $\mathcal{B} : \max\{b(y) \mid y \in Y\}$. Let $x^* \in X$ and $y^* \in Y$ such that $a(x^*) = b(y^*)$. Then x^* is optimal for $\mathcal{A}$.*

PROOF. Consider any feasible solution $x \in X$ of $\mathcal{A}$. By assumption, $\mathcal{B}$ is dual to $\mathcal{A}$, so $a(x) \geq b(y^*) = a(x^*)$, proving that x^* optimal. $\square$

If the objective value of a dual $\mathcal{B}$ coincides with the primal optimum (and thus, provides a certificate of optimality), the lower bound provided by $\mathcal{B}$ is *tight*; otherwise, we have a *duality gap*. An example arises for MINIMUM VERTEX COVER and MAXIMUM MATCHING if the graph G is bipartite (so it does not contain an odd cycle), where both optima always coincide; this is called *strong duality*. On the other hand, we may get a duality gap for non-bipartite graphs such as a triangle (for which a vertex cover requires two vertices, while a matching cannot contain more than one edge), so we only have *weak duality*.

For NP-hard optimization problems, the existence of strong duality would imply $\text{NP} = \text{coNP}$, which is considered similarly improbable as $\text{P} = \text{NP}$. In this article, we explore the concept of mutual exclusiveness, defining a clique problem (or an independent set problem on the dual graph) as a weakly dual problem to our ISP. This duality provides mathematically rigorous quality and optimality certificates for our empirical evaluation.

3 Lower Bounds: Certifying the Quality of Solutions

For a specific instance M of a minimization problem, a lower bound, $LB \in \mathbb{R}$, ensures that no solution to M can have an objective value less than LB . For an NP-hard minimization problem (such as t -ISP), one cannot expect a general method for computing tight lower bounds, as this would amount to membership in coNP . However, this does not rule out the possibility of determining good (and even tight) lower bounds for large sets of relevant benchmark instances.

In this section, we show how we can obtain such useful lower bounds for the t -ISP. A key observation is the fact that there may be some pairs of interactions $I, J \in \mathcal{I}$ that do not both occur in any valid configuration. We call such interactions *mutually exclusive*; a whole set $E \subseteq \mathcal{I}$ of interactions is mutually exclusive if all pairs within are.

The notion of “mutual exclusiveness” has previously appeared in some related literature containing SAT-based approaches for combinatorial testing [8, 97]. Given a set of t (not necessarily binary) feature variables, assigning different values to these variables results in distinct interactions. No single configuration can cover any two of these interactions simultaneously. Thereby, maximum number of such possible (valid) interactions over a fixed set of t feature variables provides a simple lower bound. While effective for nonbinary features of large domains, often offering lower bounds close to optimal sizes [8], its application to binary features is limited, yielding a maximum lower bound of 2^t . Our work extends “mutual exclusiveness” to any pair of interactions that cannot coexist in a single configuration, potentially offering improved lower bounds for both binary and nonbinary features.

Our lower bound is based on the observation that any interaction in a set of mutually exclusive and valid interactions, E , has to be part of a different configuration in any complete t -wise interaction sample. Hence, any sample needs at least $|E|$ configurations.

THEOREM 3.1. *The problem of finding a maximum-cardinality set of mutually exclusive t -wise interactions is (weakly) dual to finding a minimum-cardinality complete t -wise interaction sample.*

PROOF. We prove this by contradiction. Assume there exists a set $E \subseteq \mathcal{I}$ of mutually exclusive interactions and a complete t -wise interaction sample S such that $|E| > |S|$. Since S must cover all feasible interactions $\mathcal{I}$, including those in E , there must be a feasible configuration $C \in S$ that contains two interactions $I, J \in E$ (by the pigeonhole principle). Since I and J are mutually exclusive, C cannot be feasible, which is a contradiction and concludes the proof. $\square$

For some pairs of interactions, mutual exclusiveness can be established efficiently, e.g., if a clause from $\mathcal{D}$ is violated once all literals from $I \cup J$ are part of the configuration, or that there exist two contradicting literals $\ell \in I$ and $-\ell \in J$. In theory, deciding mutual exclusiveness is coNP-hard and can require an expensive call to a SAT solver to determine that there cannot exist a valid configuration $C \in ||\mathcal{M}||$ with $I \cup J \subseteq C$. In practice, we can mitigate this complexity, as false negatives are not critical for correctness; at worst, they may only weaken the proven bound by unnecessarily excluding interactions from the set E .

Identifying a maximum-cardinality set of mutually exclusive interactions is closely related to several well-known NP-hard problems, such as CLIQUE and INDEPENDENT SET [47]. As it is NP-hard itself, even if the complexity of detecting mutual exclusiveness is ignored, it is nontrivial to obtain good lower bounds according to the idea outlined above.

In the following, we carefully characterize different sources of mutual exclusiveness among interactions. This, in turn, describes the space of all possible sets of mutual exclusive interactions. Later we discuss how to efficiently and effectively explore such space of lower bound generating sets and close the section with an example.

3.1 Characterizing Mutual Exclusiveness

In this section, we elaborate on the notion of mutual exclusiveness of t -wise interactions and characterize different cases in which it occurs among valid interactions. For brevity, the following notation assumes $t = 2$; this does not impede the validity for other interactions (i.e., $t = 1$ or $t \geq 3$). With the aid of our characterization, the problem of finding a maximum cardinality set of mutually exclusive interactions can be formulated as a binary program, a special case of MIP. This allows us to approach the computation of good lower bounds on the sample size using MIP solvers.

Consider a pair of valid interactions I and $J \in \mathcal{I}$. I and J may be mutually exclusive, because there is an *invalid* interaction $\{p, q\}$ with $p \in I$ and $q \in J$, i.e., a combination that cannot appear together in any valid configuration ($\forall C \in ||\mathcal{M}|| : \{p, q\} \not\subseteq C$). While this is not always the case, it explains the simplest form of mutual exclusiveness that can be quickly detected. Let us capture this form as the following property.

PROPERTY 1. *A pair of valid interactions I and $J \in \mathcal{I}$ satisfying*

$$\exists p \in I \exists q \in J : \{p, q\} \notin \mathcal{I}$$

are mutually exclusive.

Mutual exclusiveness of the interactions I and J can occur even if the above property does not hold. Consider a pair I, J of interactions that do not satisfy Property 1 and a feature literal, $\ell \notin I \cup J$. If both $\{\ell\} \cup I \cup J$ and $\{-\ell\} \cup I \cup J$ include some invalid interaction, I and J are also mutually exclusive since no valid configuration including $I \cup J$ could exist containing neither $\{\ell\}$ nor $\{-\ell\}$.

The same line of reasoning can be generalized to a generic exclusiveness argument as follows. For a subset of features $F \subseteq \mathcal{F}$, let $A(F)$ be the set of all possible assignments of values to these features, with each assignment being a set of literals that contains a literal for each feature $x \in F$. For example, $A(\{x\}) = \{\{x\}, \{-x\}\}$ and $A(\{x, y\}) = \{\{x, y\}, \{-x, y\}, \{x, -y\}, \{-x, -y\}\}$.

Given a pair of valid interactions I and J , let $\overline{\mathcal{F}}_{I,J}$ denote the set of all features not yet assigned a value by the assignment $I \cup J$. Then I and J are mutually exclusive if there is a *blocking* subset of features $F \subseteq \overline{\mathcal{F}}_{I,J}$ for which any possible extension of $I \cup J$ that also assigns values to the features F contains an invalid interaction, as follows:

$$P(I, J) := \exists F \subseteq \overline{\mathcal{F}}_{I,J} \forall L \in A(F) : \exists \{p, q\} \subset L \cup I \cup J \text{ with } \{p, q\} \notin \mathcal{I}.$$

Satisfying $P(I, J)$ means that the partial assignment $I \cup J$ cannot be extended to a valid configuration. However, there may still be some cases of mutual exclusiveness that cannot be discovered by predicate $P(I, J)$. To capture these additional cases, consider the situation in which predicate $P(I, J)$, as defined above, is false. It means that there is an assignment $L \in A(\overline{\mathcal{F}}_{I,J})$, such that all t -wise interactions in $I \cup J \cup L$ are valid. We can extend $P(I, J)$ to a predicate $D(I, J)$ that recognizes all cases of mutual exclusiveness by checking whether all such extensions $I \cup J \cup L$ are invalid configurations. Note that the predicates $P(I, J)$ and $D(I, J)$ consider a large number of cases and can thus be costly to evaluate. However, weaker forms of $P(I, J)$ can be evaluated by only searching for blocking sets F of limited size, e.g., of size 0 (resulting in Property 1), 1 or 2, which can be done efficiently. Now we can formulate the problem of finding a maximum cardinality set of mutually exclusive interactions as a binary program:

$$\max \quad \sum_{I \in \mathcal{I}} x_I \quad (1)$$

$$\forall I, J \in \mathcal{I} \text{ with } \neg Q(I, J) : \quad x_I + x_J \leq 1, \quad (2)$$

$$\forall I \in \mathcal{I} : \quad x_I \in \{0, 1\}. \quad (3)$$

Here $Q(I, J)$ is a predicate indicating mutual exclusiveness of interactions I, J , e.g., $D(I, J)$ or, more practically, a restricted version of $P(I, J)$ such as Property 1. In this formulation, the binary variable x_I decides whether the interaction I is included in a desired set of mutually exclusive interactions. Equation (1) maximizes the number of interactions in the desired set, while Equation (2) enforces mutual exclusiveness for any pair of selected interactions. Here the set of all valid interactions, $\mathcal{I}$, can efficiently be extracted from a given feasible sample. For any set of valid interactions $\mathcal{I}'$, let $\text{OptLB}(\mathcal{I}')$ denote the set of all interactions selected by the binary program.

In practice, the above binary program can be very hard to solve, mainly due to the large number of valid interactions $\mathcal{I}$ in many practically relevant instances. Therefore, we consider restricting the above predicates and the binary program to a smaller subset of valid interactions $\mathcal{I}' \subseteq \mathcal{I}$. By using the above binary program—with Equation (2) generated by Property 1—to optimize over different subsets of valid interactions, we can construct a LNS algorithm that is very effective. We elaborate on the details of this algorithm later in this section.

3.2 Extended Use of Invalid Interactions

The role of invalid interactions is not limited to detecting mutual exclusiveness of valid interactions. Invalid interactions also suggest a constructive search pattern for heuristically finding large sets of mutually exclusive interactions as follows.

Consider a set of mutually exclusive interactions E , and a feature literal ℓ . The set E is called *feature fixed* at ℓ , denoted as E_ℓ , if all interactions I in E contain ℓ , i.e., $\forall I \in E : \ell \in I$. Feature-fixed sets of mutually exclusive interactions are of interest, because Property 1 immediately provides the following.

PROPERTY 2. *For any invalid interaction $\{p, q\}$, any feature-fixed sets E_p and E_q are disjoint and $E_p \cup E_q$ is mutually exclusive.*

Starting at a feature literal p and having a feature-fixed E_p computed, this property suggests considering any possible invalid interaction $\{p, q\}$ to extend E_p by adding interactions from any E_q .

As a simple example, consider three features $\mathcal{F} = \{1, 2, 3\}$ and no constraints between them. Thus, the only invalid interactions are of the form $\{i, -i\}$ for each $i \in \mathcal{F}$. According to Property 2, one could simply start with $E_1 = \{\{1, 2\}, \{1, -2\}\}$, a mutually exclusive set of interactions fixed at feature 1, and then be guided by an invalid interaction like $\{1, -1\}$ to safely report $E_1 \cup E_{-1}$ as a bigger set of mutually exclusive interactions. Here this new set could be

Algorithm 1: LB-LNS

```

 $E = \text{LBSEARCH}()$  [Initial set of mut. excl. interactions]
while within time limit do
  Select  $E' \subseteq E$  [The part of the set we remove]
   $I' =$  valid interactions that are mut. excl. to  $E \setminus E'$ 
   $E'' = \text{OPTLB}(I')$  [With short time limit]
   $E = (E \setminus E') \cup E''$  [Combine to new set]
  Tune  $E'$ -selection based on difficulty of computing  $E''$ 
end while
return  $E$ 

```

$E = \{\{1, 2\}, \{1, -2\}, \{-1, 2\}, \{-1, -2\}\}$; it is easy to verify that these interactions are mutually exclusive. Furthermore, the sample $S = \{\{1, -2, 3\}, \{-1, -2, -3\}, \{1, 2, -3\}, \{-1, 2, 3\}\}$ covers all valid pairwise interactions with four configurations. Hence, for this example, it gives us a matching lower and upper bound on the number of configurations in a sample, showing that both E and S have optimum cardinality.

To obtain large mutually exclusive sets of interactions by combining feature-fixed sets, it is sensible to focus on large feature-fixed sets E_p, E_q . Given a feature literal ℓ , a maximum cardinality E_ℓ could be obtained by a simple adaptation of the above-defined binary program. From another point of view, any maximum cardinality set of mutually exclusive interactions can be written as the union of carefully selected feature-fixed E_ℓ 's over all ℓ .

Now we describe an algorithm called LBSEARCH to search for large sets of mutually exclusive interactions. We start with an empty set E and repeat the following steps over each feature literal p . We first construct the maximum cardinality E_p and try to add it to E . For any invalid interaction $\{p, q\}$, we then construct the maximum cardinality E_q and try to add E_q to E . During the above two steps, it may be impossible to add some interaction I to E because I may not be mutually exclusive with some other interaction J already in E . In this case, the conflicting interaction $J \in E$ is recorded. After a prespecified number of adding operations, with some fixed probability, either the highest conflicting portion of E is removed, or E is emptied.

3.3 LNS for Lower Bounds

While LBSEARCH often yields large sets of mutually exclusive interactions, it can still be improved. In the following, we describe how OPTLB(I'), which solves the binary program defined by Equations (1) to (3), can be utilized in a LNS to obtain even larger sets.

LB-LNS (Algorithm 1) starts with an initial solution from the lower bound heuristic in Section 3.2. Subsequently, it iteratively removes some part E' of the current best solution and replaces it with an optimal solution from OPTLB. Computing the candidate interactions I' , i.e., the set of all interactions that are mutually exclusive to the remaining elements $E \setminus E'$, can be done efficiently during the selection of E' . At the end of each iteration, a new solution is constructed by taking the union of $E \setminus E'$ with the output of OPTLB(I'). The formal description is given in Algorithm 1 and an example can be found in Section 3.4.

The most challenging part of the algorithm is the selection of interactions to remove from a current solution. The size of the remaining interactions I' must be small enough so that OPTLB can still solve it, but large enough to yield any improvement for the current best solution. Thus, the selection procedure for E' is adapted according to the difficulty of the integer program subroutine. That is choosing a smaller or larger E' for the next iteration.

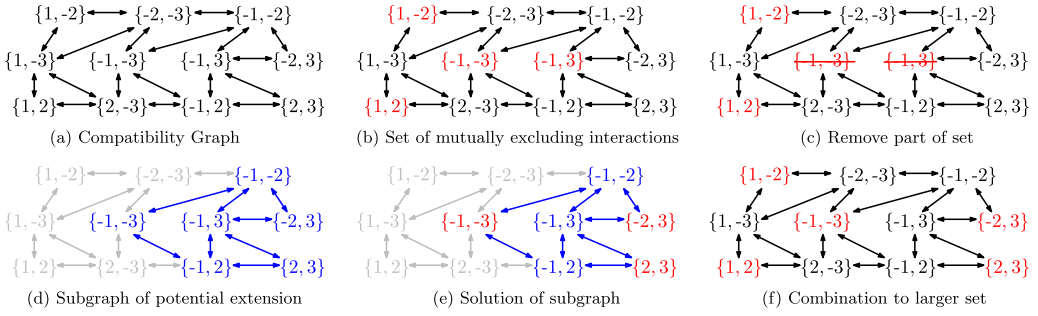


Fig. 1. Example of the lower bound computation with Algorithm 1. An edge in the compatibility graph indicates that the two interactions are compatible and can appear in the same configuration. A set of interactions without any edges, like in (b), is mutually exclusive and a lower bound on the necessary number of configurations. In this example, the initial set of mutually exclusive tuples $\{\{1, 2\}, \{1, -2\}, \{-1, 3\}, \{-1, -3\}\}$ increases to $\{\{1, 2\}, \{1, -2\}, \{-1, -3\}, \{2, 3\}, \{-2, 3\}\}$.

3.4 Example

For a small example of Algorithm 1, consider a mutually exclusive set of interactions with the features $\mathcal{F} = \{1, 2, 3\}$ and a single clause $\mathcal{D} = \{\{-1, -3\}\}$. The *Compatibility Graph*, as shown in Figure 1(a), has a vertex for each valid interaction and edges for pairs of interactions that can appear in the same configuration. As a consequence, we are looking for an independent set, i.e., a subset of vertices without any edges in between. We start with a randomly obtained initial set of mutually exclusive interactions $E = \{\{1, 2\}, \{1, -2\}, \{-1, 3\}, \{-1, -3\}\}$ as highlighted in red in Figure 1(b). We can see that it is indeed mutually exclusive as it is an independent set without any edges between its elements. Now perform an iteration of LNS, by (randomly) selecting $E' = \{\{-1, 3\}, \{-1, -3\}\}$ as the subset to be removed, see Figure 1(c). This gives us the smaller subproblem with $I' = \{\{-1, 2\}, \{-1, -2\}, \{-1, 3\}, \{-1, -3\}, \{2, 3\}, \{-2, 3\}\}$, see Figure 1(d). This subproblem consists of all interactions that are mutually exclusive with the whole remaining set; usually this strongly reduces the number of vertices. We compute on I' an optimal mutually exclusive set $E'' = \{\{-1, -3\}, \{2, 3\}, \{-2, 3\}\}$, see Figure 1(e). Because I' is mutually exclusive to the remaining set $E \setminus E'$, we can combine $(E \setminus E') \cup E''$ to a new, larger set, see Figure 1(f). This could be repeated; in the example, we have already reached optimality.

4 Computing (Near-)Optimal Solutions

In the previous section, we explored the computation of large sets of mutually exclusive interactions, which provide lower bounds on the achievable sample size but do not yield the samples themselves. In this section, we focus on minimizing the actual sample size.

Both the lower bound computation and the sample minimization will be executed in parallel. These processes will continue until they converge to the same value, indicating an optimal sample size, or until a specified time limit is reached. Due to the inherent duality of these problems, we can still bound the optimality gap, even if convergence is not achieved within the time limit.

We begin by discussing the computation of optimal solutions for small instances, which we achieve by modeling the problem as a (CP-)SAT formula. This formulation is then used as a subroutine to compute (near-)optimal solutions for larger instances through a LNS approach. In LNS, we iteratively solve sufficiently small parts of a larger instance to optimality, progressively improving the overall solution quality.

4.1 Modeling Pairwise Sampling in CP-SAT

Finding a minimal sample to cover all valid interactions $\mathcal{I}$ can be described by a set of Boolean and linear constraints that can be solved by the CP solver CP-SAT. This does not scale well, but is a valuable subroutine in the later algorithm.

Let $x_1, x_2, \dots, x_n \in \mathbb{B}$ represent the feature selection. Recall that a clause $\mathcal{D}_j$ is a set of (negated or unnegated) literals, indicating the desired feature values. For ease of notation we denote for any literal $\ell \in \mathcal{D}_j$, x_ℓ to be the (possibly negated) representation of the feature $|\ell|$, i.e., $x_\ell = \overline{x_{|\ell|}}$ if $\ell < 0$. We introduce further variables $y_I \in \mathbb{B}$ for all interactions $I \in \mathcal{I}$, which can only be true if the interaction I is covered by the assignments of $x_1, x_2, \dots, x_n$. For an interaction I , this can be enforced by adding implications $y_I \implies \bigwedge_{\ell \in I} x_\ell$.

The idea of our CP-SAT formulation is to use k copies of variables and constraints, where k is an upper bound on the expected number of configurations. If an estimate for k is too low, the model becomes infeasible, and the solution process can be retried for increased k .

We can find a feasible sample with at most k configurations by enforcing that for each interaction I , at least one of the k copies of y_I must be true. For each of the k copies, we introduce a further Boolean variable $u_i \in \mathbb{B}$ that indicates if the copy is used to represent a sample configuration. To minimize the number of sample configurations required to cover all interactions, we can then use the objective to minimize the number of copies used:

$$\min \sum_{i=1}^k u_i \quad (4)$$

$$\forall i = 1, \dots, k, I \in \mathcal{I} : \overline{u_i} \implies \overline{y_I^i} \quad (5)$$

$$\forall i = 1, \dots, k, \mathcal{D}_j \in \mathcal{D} : \bigvee_{\ell \in \mathcal{D}_j} x_\ell^i \quad (6)$$

$$\forall i = 1, \dots, k, I \in \mathcal{I} : y_I^i \implies \bigwedge_{\ell \in I} x_\ell^i \quad (7)$$

$$\forall I \in \mathcal{I} : \bigvee_{i=1}^k y_I^i. \quad (8)$$

Here Equation (4) minimizes the sample size, i.e., the number of active copies (indicated by $u_i \in \mathbb{B}$), Equation (5) prevents deactivated copies of sample configurations from covering interactions, Equation (6) ensures sample configurations are valid according to the feature model, Equation (7) forces y_I^i to be false if the interaction I is not covered in the i th configuration, and Equation (8) ensures each interaction $I \in \mathcal{I}$ to be covered by at least one configuration.

For simplicity, we restrict ourselves to CNF; the solver CP-SAT actually allows complex constraints with more efficient formulations. We use these more efficient constraints whenever possible in the actual implementation, especially for modeling the common “alternative”-constraint. However, if the instance can actually be expressed efficiently in CNF, using a SAT solver with iterative search for optimizing the objective, as done by Yamada et al. [97], may be more efficient than CP-SAT if the model only utilizes a CNF representation. While the formulation can theoretically also be solved with Integer Programming, the strong Boolean logic of the problem makes it more suitable for SAT-based solvers as they are specifically designed for this.

A serious issue with this formulation is its high symmetry due to the copied parts: Every solution, feasible or infeasible, has a number of representations exponential in k by permuting the assignments of $x_1^i, \dots, x_n^i$ over i . This difficulty for the solver is common, e.g., also for GRAPH COLORING, for which permuting the colors yields an exponential number of different but logically

equivalent solutions (cf. [59]). We can break these symmetries and greatly improve the performance of the solver by computing a set of mutually exclusive interactions E using Algorithm 1 and enforce each interaction of that set to be covered in a specific copy i :

$$\forall I \in E : y_I^i = \text{true}. \quad (9)$$

In the following, the computed optimal sample is denoted by $\text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, \mathcal{I}, k)$. If there is a time limit, it will return the best solution found, or $\perp$ if no sample that covers all interactions in $\mathcal{I}$ has been found.

When an initial solution is available—whether precomputed or quickly obtained through one of the many available heuristics—it can significantly assist OPTSAMPLE . This initial solution serves multiple purposes: It provides a sufficient value for k , offers a warm start for the solver, and acts as a fallback, ensuring that a feasible solution is always returned, even if OPTSAMPLE does not terminate within the allotted time.

With this approach, most of our benchmark models with fewer than 20 features could be solved directly to provable optimality. For larger models, the formulation size of $\Omega(n \times k + |\mathcal{I}| \times k)$ becomes quickly prohibitive (mostly due to the quickly growing $|\mathcal{I}|$).

4.2 LNS

The preceding ideas can be extended to compute (near-)optimal solutions for even larger configurable systems. Specifically, if we already have a sample, we can use OPTSAMPLE to optimize a large portion of it by a significantly smaller, practically tractable optimization model. This approach allows us to efficiently explore parts of interest in the solution space with each iteration, leading to rapid convergence toward a (near-)optimal solution.

Although Equation (6) is the most complex constraint, it can generally be solved quickly, as demonstrated by algorithms like YASA [57], ICPL [44], IncLing [4], and Chvátal's algorithm [43]. However, the crucial challenge lies in managing the number of additional constraints that arise from the large set of valid interactions $\mathcal{I}$. To keep the optimization model in OPTSAMPLE practically tractable, we need to reduce the size of $\mathcal{I}$.

An important observation is that the coverage of $\mathcal{I}$ increases rapidly with each additional configuration, while only a small amount of coverage is lost even when multiple configurations are removed from a sample. This behavior is illustrated for some concrete configurable systems in Figure 2.

If we remove a subset $S' \subset S$ from a sample S , uncovering the interactions $\mathcal{I}' \subseteq \mathcal{I}$, we can use $\text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, \mathcal{I}', |S'|)$ to repair the sample with as few configurations as possible. For instance, in the FreeBSD-8_0_0 sample shown in Figure 2, removing 50% of the sample results in a loss of only about 0.3% of the coverage. Consequently, we can apply $\text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, \mathcal{I}', |S'|)$ to find the best replacement for 50% of the sample, using an optimization model that is only roughly 0.3% the size of the full problem. This approach still provides significant optimization potential while drastically reducing the problem size.

We incorporate this idea into an algorithm (Algorithm 2) that iteratively removes parts of an initial solution and repairs it optimally. An initial sample can be quickly computed using YASA [57] with the parameter $m = 1$.

A performance-critical part of Algorithm 2 is the size of S' , the configurations that are removed from the current sample S (similar to the selection of E' in Algorithm 1). This needs to be sufficiently small, such that $\text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, \mathcal{I}', k)$ can compute an optimal (or at least a good) solution. At the same time, it should be as large as possible to actually achieve improvements. The difficulty of the resulting subproblem is hard to predict, so the selection should be dynamic and increase the size if the iterations are very fast, or decrease its size if the iterations are reaching the time limit.

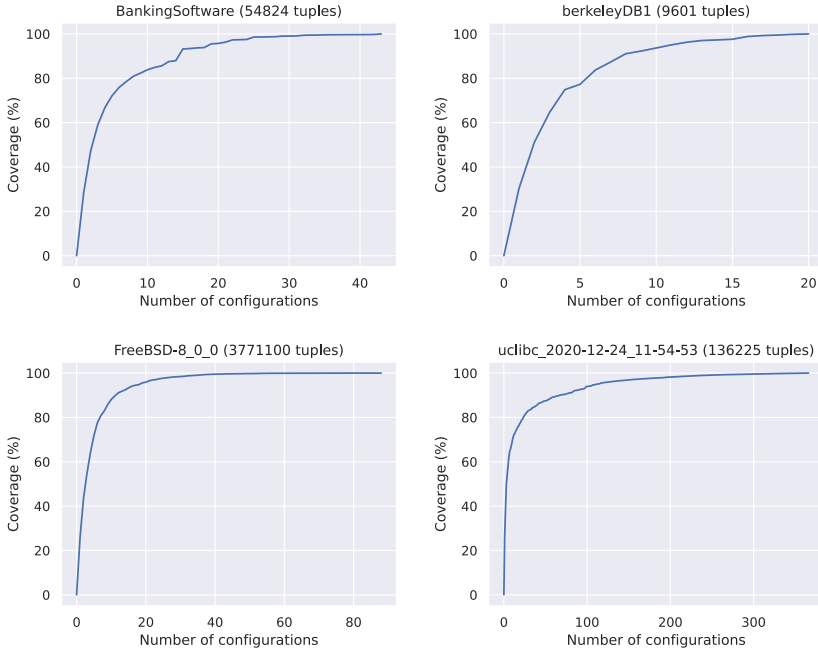


Fig. 2. Examples of the coverage over the number of configurations. Each plot illustrates how coverage increases with the number of configurations: For instance, a value of 90% at 10 indicates that the first 10 configurations in a randomly ordered sample would cover 90% of all feasible interactions. These samples were computed using YASA [57]. The rapid growth in coverage shows that even removing a large fraction of configurations leaves only a minor portion of interactions uncovered, which is crucial for the size of our constraint program. With few interactions becoming uncovered, the task of optimally recovering these through the constraint program becomes manageable. By shuffling the sample, we can repeatedly explore replacing large subsets of the sample with minimal configurations to achieve equivalent coverage.

Algorithm 2: SampLNS

```

 $S$  = initial sample computed by YASA ( $m = 1$ )
Start LB-LNS on a separate thread
while within time limit and  $|S| >$  lower bound do
    Select  $S' \subseteq S$  [The part of the solution we remove]
     $I'$  = interactions covered by  $S'$  but not by  $S \setminus S'$ 
     $k = |S'|$  [Upper bound on covering  $I'$ ]
     $S'' = \text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, I', k)$  [short time limit]
     $S = (S \setminus S') \cup S''$  [Combine to new solution]
    Tune  $S'$ -selection based on difficulty of computing  $S''$ 
end while
Terminate LB-LNS and retrieve latest  $E$ 
return optimized sample  $S$ , lower bound  $|E|$ 
  
```

As previously mentioned, OPTSAMPLE can be supported by providing an initial solution, ensuring that it will either return this solution or find a better one. We use the removed part, S' , as the initial solution, which guarantees that the new solution will either be an improvement upon or revert to S , since $(S \setminus S') \cup S' = S$.

4.3 Example

For the benefit of the reader, we provide an iteration of the algorithm for a simple example. Consider an instance with features $\mathcal{F} = \{1, 2, 3, 4\}$ and two clauses $\mathcal{D} = \{\{1, 2\}, \{3, 4\}\}$. This instance has 22 valid pairwise interactions $\mathcal{I}$ that need to be covered:

$$\mathcal{I} = \{\{3, 4\}, \{1, -3\}, \{2, -4\}, \{1, 3\}, \{-2, 4\}, \{-1, 4\}, \{2, 4\}, \{1, 2\}, \{1, -4\}, \{-2, -3\}, \{-1, -3\}, \{-2, 3\}, \\ \{-1, 3\}, \{3, -4\}, \{-3, 4\}, \{2, -3\}, \{1, -2\}, \{1, 4\}, \{2, 3\}, \{-1, -4\}, \{-2, -4\}, \{-1, 2\}\}$$

Assume that YASA has provided the following initial sample of size six:

$$S = \{\{1, 2, -3, 4\}, \{1, -2, 3, -4\}, \{1, -2, -3, 4\}, \{-1, 2, 3, 4\}, \{-1, 2, 3, -4\}, \{-1, 2, -3, 4\}\}.$$

By looking at the interactions $\{1, 2\}$, $\{1, 3\}$, $\{-2, -3\}$, $\{3, 4\}$, $\{-1, -4\}$, and $\{-1, -3\}$, it is straightforward to check that no configuration can be removed without missing an interaction, so there is no trivial improvement. Now consider the three configurations

$$S' = \{\{1, 2, -3, 4\}, \{-1, 2, 3, 4\}, \{-1, 2, 3, -4\}\}$$

for possible removal. This would leave the interactions

$$\mathcal{I}' = \{\{1, 2\}, \{2, 3\}, \{3, 4\}, \{2, -4\}, \{-1, 3\}, \{-1, -4\}\}$$

uncovered, for which we seek a cover with fewer configurations. Solving $\text{OPTSAMPLE}(\mathcal{F}, \mathcal{D}, \mathcal{I}', |S'|)$ gives us an optimal sample $S'' = \{\{1, 2, 3, 4\}, \{-1, 2, -3, 4\}\}$ for $\mathcal{I}'$, which is smaller than the part we removed. Notice how we removed half of the configurations, but only had to find a new sample for 6 out of the 22 interactions. Figure 2 shows that this effect is even stronger for real instances, explaining the effectiveness of our approach. Combining yields a better sample for all interactions

$$S = (S \setminus S') \cup S'' = \{\{1, -2, 3, -4\}, \{1, -2, -3, 4\}, \{-1, 2, -3, 4\}, \{1, 2, 3, 4\}, \{-1, 2, -3, 4\}\}.$$

This is optimal for the example, but could be continued if necessary; in that case, keeping track of the remaining computing time allows adjustment by selecting a smaller or larger S' for the next iteration.

5 Further Improvements

The practical implementation can be tuned in various ways.

Instance Encoding and Preprocessing. We accept feature models in the XML-format of FeatureIDE [69], consisting of a feature hierarchy and a set of cross-tree constraints in propositional logic. This instance is simplified by some simple rules, e.g., replacing trivial clauses with only one element, or substituting literals with a bidirectional implication by the same variable. The feature hierarchy also defines the set of concrete features, whose interactions have to be covered. Additionally, we also accept the DIMACS format of SAT solvers, which is a CNF formula with a set of variables and clauses. This format is not preprocessed and all variables are considered to be concrete features.

Selecting $E' \subseteq E$ in Algorithm 1. We need to select $E' \subseteq E$ in Algorithm 1, such that we can still solve OPTLB, but as large as possible to escape deeper local optima. For this, we start with $E' = E$ and iteratively remove random interactions from it, until $|I'|$ is below a threshold $\gamma = 1,000$. Interactions that would result in $I' = \emptyset$ are excluded from removal. To scale the threshold γ automatically to the right size, it is increased by 10% if an iteration uses less than 50% of its time, and decreased by 10% if it uses more than 95% of its time limit.

Selecting $S' \subseteq S$ in SampLNS. We use an analogous strategy for Algorithm 2. Starting with $S' = \emptyset$, we repeatedly select a random configuration from $S \setminus S'$ and add it to S' , as long as the number of missing interactions in $S \setminus S'$ remains below a threshold ϕ . Initially ϕ is set to 250, but we increase this value by 25% if OPTSAMPLE computes an optimal solution and decrease it again by 25% if OPTSAMPLE is not able to compute an optimal solution or improve the sample. Due to its exponential character, this simple trick allows us to automatically scale to the largest feasible size of S' within a few iterations.

Symmetry Breaking in OPTSAMPLE. At the beginning of each iteration in Algorithm 2, we compute a set of mutually exclusive interactions on I' for breaking symmetries in OPTSAMPLE, or skipping the optimization if we notice that no improvement is possible. For this, we first compute a simple greedy solution (sorting based on the number of appearances in a sample, as interactions with fewer appearances are more likely to be exclusive) and then optimize it with Algorithm 1 for up to 10% of the iteration time limit.

Parallelization. Our implementation of SampLNS will spawn a thread running Algorithm 1 (single threaded) and use all remaining cores to optimize the sample (CP-SAT can make use of multiple cores). After each iteration of SampLNS, the latest lower bound of Algorithm 1 is queried (used to check for premature termination due to optimality). For smaller feature models, OPTSAMPLE may yield better lower bounds than Algorithm 1, such that the lower bounds are in rare cases not actually computed by Algorithm 1. The thread of Algorithm 1 is independent of the additional use of Algorithm 1 for symmetry breaking at the beginning of each iteration.

6 Empirical Evaluation

In the preceding sections, we presented a new parametric algorithm called SampLNS that is able to minimize samples and compute provable lower bounds for the achievable sample size. In this section, we investigate the quality of the lower bounds and the minimization potential empirically. We designed and performed an experiment to answer the following four **Research Questions (RQs)**:

- RQ1. Can existing algorithms compute samples with a sample size equal or close to the lower bound?
- RQ2. Can SampLNS compute samples of smaller size than existing algorithms?
- RQ3. What is the computational effort to derive lower bounds and optimize samples with SampLNS?
- RQ4. Does SampLNS depend on the sample quality of YASA?

6.1 Experimental Design

We used feature models from multiple sources that originate from different domains, namely finance, systems software, e-commerce, gaming, and communication [53, 69, 77, 79, 85] and encompass a wide range of number of features (9–1,408) and number of constraints (13–15,692) (cf. Table 1). For a wide range of feature model sizes, we selected small- and medium-sized feature models

Table 1. Comparison of SampLNS with Algorithms from the Literature

Feature Model	$ \mathcal{F} $	$ \mathcal{D} $	Baseline <i>Min</i>	SampLNS UB <i>Mean (Min)</i>	SampLNS LB <i>Mean (Max)</i>	Savings	SampLNS UB/LB	Time to Bounds
calculate	9	15	9	5 (5)	5 (5)	44% (44%)	1.00 (1.00)	<1 s (1 s)
lcm	9	16	8	6 (6)	6 (6)	25% (25%)	1.00 (1.00)	<1 s (<1 s)
email	10	17	6	6 (6)	6 (6)	0% (0%)	1.00 (1.00)	<1 s (<1 s)
ChatClient	14	20	7	7 (7)	7 (7)	0% (0%)	1.00 (1.00)	1 s (2 s)
toybox_2006-10-31...	16	13	9	8 (8)	8 (8)	11% (11%)	1.00 (1.00)	1 s (1 s)
car	16	33	6	5 (5)	5 (5)	17% (17%)	1.00 (1.00)	<1 s (<1 s)
FeatureIDE	19	27	9	8 (8)	8 (8)	11% (11%)	1.00 (1.00)	271 s (128 s)
FameDB	22	40	8	8 (8)	8 (8)	0% (0%)	1.00 (1.00)	1 s (1 s)
APL	23	35	9	7 (7)	7 (7)	22% (22%)	1.00 (1.00)	1 s (1 s)
SafeBali	24	45	11	11 (11)	11 (11)	0% (0%)	1.00 (1.00)	<1 s (<1 s)
TightVNC	28	39	11	8 (8)	8 (8)	27% (27%)	1.00 (1.00)	16 s (21 s)
APL-Model	28	40	10	8 (8)	8 (8)	20% (20%)	1.00 (1.00)	14 s (15 s)
gpl	38	99	17	16 (16)	16 (16)	5.9% (5.9%)	1.00 (1.00)	3 s (3 s)
SortingLine	39	77	12	9 (9)	9 (9)	25% (25%)	1.00 (1.00)	8 s (9 s)
dell	46	244	32	31 (31)	31 (31)	3.1% (3.1%)	1.00 (1.00)	29 s (45 s)
PPU	52	109	12	12 (12)	12 (12)	0% (0%)	1.00 (1.00)	2 s (2 s)
berkeleyDB1	76	147	19	15 (15)	15 (15)	21% (21%)	1.00 (1.00)	77 s (137 s)
axTLS	96	183	16	11 (11)	10 (10)	31% (31%)	1.10 (1.10)	20 s (20 s)
Violet	101	203	23	17 (17)	16 (16)	26% (26%)	1.06 (1.06)	476 s (656 s)
berkeleyDB2	119	346	20	12 (12)	12 (12)	40% (40%)	1.00 (1.00)	162 s (282 s)
soletta_2015-06-2...	129	192	30	24 (24)	24 (24)	20% (20%)	1.00 (1.00)	21 s (60 s)
BattleofTanks	144	769	451	320 (295)	256 (256)	29% (35%)	1.25 (1.15)	887 s (160 min)
BankingSoftware	176	280	40	29 (29)	29 (29)	28% (28%)	1.00 (1.00)	306 s (429 s)
fiasco_2017-09-26...	230	1,181	234	225 (225)	225 (225)	3.8% (3.9%)	1.00 (1.00)	382 s (579 s)
fiasco_2020-12-01...	258	1,542	209	196 (196)	196 (196)	6.1% (6.2%)	1.00 (1.00)	438 s (478 s)
uclibc_2008-06-05...	263	1,699	505	505 (505)	505 (505)	0% (0%)	1.00 (1.00)	104 s (67 s)
uclibc_2020-12-24...	272	1,670	365	365 (365)	365 (365)	0% (0%)	1.00 (1.00)	108 s (112 s)
E-Shop	326	499	19	12 (12)	9 (10)	37% (37%)	1.30 (1.20)	268 s (64 min)
toybox_2020-12-06...	334	92	18	13 (13)	7 (8)	28% (28%)	1.71 (1.62)	532 s (35 min)
DMIE	366	627	26	16 (16)	16 (16)	38% (38%)	1.00 (1.00)	104 s (135 s)
soletta_2017-03-0...	458	1,862	56	37 (37)	31 (37)	34% (34%)	1.16 (1.00)	387 s (24 min)
busybox_2007-01-2...	540	429	34	21 (21)	21 (21)	38% (38%)	1.00 (1.00)	164 s (237 s)
fs_2017-05-22	557	4,992	398	396 (396)	396 (396)	0.5% (0.5%)	1.00 (1.00)	478 s (575 s)
WaterlooGenerated	580	879	144	82 (82)	82 (82)	43% (43%)	1.00 (1.00)	223 s (310 s)
financial_services	771	7,238	4384	4,368 (4,340)	4,274 (4,336)	0.36% (1%)	1.02 (1.00)	862 s (102 min)
busybox-1_18_0	854	1,164	26	16 (16)	11 (13)	35% (38%)	1.53 (1.23)	233 s (59 min)
busybox-1_29_2	1,018	997	36	22 (22)	17 (21)	38% (39%)	1.26 (1.05)	465 s (60 min)
busybox_2020-12-1...	1,050	996	33	21 (20)	17 (19)	36% (39%)	1.19 (1.05)	407 s (17 min)
am31_sim	1,178	2,747	60	36 (33)	26 (29)	39% (45%)	1.36 (1.14)	699 s (77 min)
EMBTToolkit	1,179	5,414	1881	1,879 (1,872)	1,821 (1,872)	0.1% (0.48%)	1.03 (1.00)	863 s (47 min)
atlas_mips32_4kc	1,229	2,875	66	38 (36)	31 (33)	41% (45%)	1.22 (1.09)	548 s (50 min)
eCos-3-0_i386pc	1,245	3,723	64	43 (39)	31 (36)	32% (39%)	1.38 (1.08)	621 s (146 min)
integrator_arm7	1,272	2,980	66	38 (36)	30 (33)	41% (45%)	1.28 (1.09)	681 s (82 min)
XSEngine	1,273	2,942	63	38 (36)	31 (32)	39% (43%)	1.23 (1.12)	572 s (52 min)
aaed2000	1,298	3,036	87	55 (52)	51 (51)	36% (40%)	1.09 (1.02)	707 s (75 min)
FreeBSD-8_0_0	1,397	15,692	76	47 (41)	27 (30)	38% (46%)	1.72 (1.37)	831 s (120 min)
ea2468	1,408	3,319	65	38 (36)	31 (32)	41% (45%)	1.24 (1.12)	721 s (67 min)
Optimality			7	≥ 26				
			[15%]	[55%]				
Improvements						40 [85%]		

Here, "Baseline" indicates the best sample size found by any of the existing algorithms, each having five runs of 15 min. "SampLNS UB" shows the sample size found by SampLNS with the initial numbers indicating the mean over five runs of 15 min each, i.e., typical outcomes for a short run, while the numbers in parentheses describe the best of five extended runs of up to 3 h, i.e., the potential outcomes of longer runs. Similarly, "SampLNS LB" is the lower bound, while "Savings" quantifies the reduction in sample size achieved by SampLNS compared to previous algorithms. "SampLNS UB/LB" illustrates the relationship between upper and lower bounds, with a value of 1.0 implying optimality. "Time to Bounds" records the moment when final lower and upper bounds were established, potentially preceding the time limit. A low value may signal efficiency, yet it could also suggest stagnation, depending on the solution quality at that moment. Optimal values are additionally highlighted in bold.

from examples provided by the tool FeatureIDE [69]. We also used feature models from real-world Kconfig systems, provided by Pett et al. [77], for which we chose the earliest and latest versions for each system. In addition, we used more complex, real-world feature models [53, 79, 85]. Knüppel et al. [53] provide 116 different models for the eCos system. Most of these models have similar properties, so we only used six models with different numbers of features.

To investigate the performance of existing algorithms and SampLNS, we selected multiple state-of-the-art algorithms for t -wise interaction sampling and compared their performance on the benchmark models. In particular, we selected the following algorithms: Chvátal [21, 43], ICPL [44],

IPOG [60], IncLing [4], and YASA [57]. All of these are well-known greedy algorithms with different sampling strategies and were also used in previous evaluations [4, 7, 44, 57]. Especially ICPL is regarded as one of the best t -wise sampling algorithms and was used as benchmark for many other algorithms that we did not include in this evaluation [31, 37, 60, 74]. Some other algorithms from previous evaluations either lack publicly available implementations or are incompatible with our Linux workstations [24, 31, 35, 73, 76, 86], and therefore could not be included in our comparison. However, based on prior evaluations, these missing algorithm are not expected to be competitive.

As our problem specification requires 100% pairwise coverage, we have chosen only algorithms that are guaranteed to generate such samples. This excludes, for instance, algorithms that generate random samples [72] and most algorithms that employ local or population-based search [22, 27, 37, 38].

YASA provides a parameter m for decreasing the sample size through multiple sampling iterations. We generate samples with YASA using multiple values for this parameter, in particular, 1, 3, 5, and 10. To ensure a more equitable comparison with SampLNS, which utilizes the full time limit for refinement, we also implemented a variant of YASA that, instead of being constrained by a fixed number of sampling iterations m , performs as many iterations as possible within the given time limit.

We conducted three experiments as follows:

- (1) First, we establish provable lower bounds for each model by executing SampLNS five times with a generous time limit of 3 h (180 s for each optimization step) and using the best bounds obtained. These lower bounds were subsequently employed in the other experiments to estimate the optimality gap.
- (2) Each of the previously mentioned algorithms, as well as SampLNS, was run five times for each model with a time limit of 900 s (60 s for each optimization step). The mean of the results was used to compare the performance of these algorithms. For SampLNS, this time limit includes the runtime of YASA for obtaining the initial sample to ensure a fair comparison, even though YASA ($m = 1$) typically completes in just a few seconds.
- (3) SampLNS was additionally run with four different algorithms for computing the initial sample, five times for each model, with a time limit of 900 s (60 s for each optimization step). This experiment aimed to assess the impact of the initial sample on the performance of SampLNS. Importantly, the runtime of the initial sample computation was this time not counted within the time limit, ensuring that SampLNS had an equal opportunity for each initial sample.

All experiments were executed on AMD Ryzen 9 7900 with 96 GB DDR5 RAM and Ubuntu 22.04. The code was written in Python 3.12 with native C++-17 elements using PyBind11 (v2.10.3), compiled by g++ (v11.3.0). We used CP-SAT of or-tools (v9.8) and the MIP solver Gurobi (v11.0). Code and data are available online (<https://github.com/tubs-alg/SampLNS>). For the baseline algorithms and YASA, we use the FeatureIDE library, written in Java, which provides a uniform API to several sampling algorithms [6, 55].

6.2 Results

6.2.1 (RQ1) Can Existing Algorithms Compute Samples with a Sample Size Equal or Close to the Lower Bound. The size of the smallest sample found by any existing algorithm for each feature model is listed in the column *Baseline* of Table 1. The best lower bound found by SampLNS is listed in parentheses in the column *SampLNS LB*, and the column *SampLNS UB* shows in parentheses the sample size of the smallest known sample (all achieved by SampLNS), defining the range in

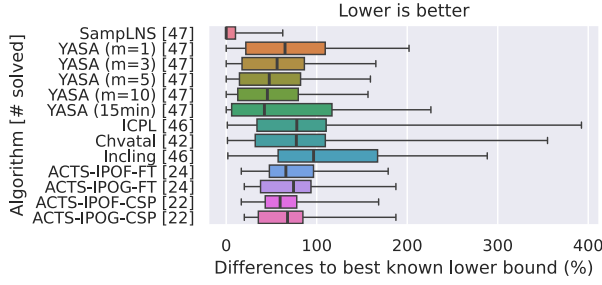


Fig. 3. Differences of the sample sizes of various sampling algorithms (with a 900 s time limit) to the best lower bound computed by SampLNS. Not all algorithms were able to compute a feasible sample within the time limit, thus, the number of successfully solved models is added in parentheses. We can see that SampLNS never timed out, and 75% of the samples are at most 10% above the lower bound. More than 55% of the samples match the lower bound and, thus, are minimal. The next best algorithm is YASA with $m = 10$ (respectively, the 15 min-variant) with a median difference of 46% (respectively, 42%). IncLing yields the largest samples, with a median of twice the size of the lower bound.

which the smallest possible sample can lie. For 7 of the 47 feature models, the smallest sample found by any existing algorithm is equal to the lower bound. For at least 28 feature models, the difference to the optimum is above 20%. Figure 3 displays the differences to the lower bounds for each algorithm separately. A value of 20% would imply that the sample size is $1.2\times$ the lower bound. We can see that of the previous algorithms, YASA with $m = 10$ (respectively, the 15 min-variant) achieves the smallest samples, with a median of 46% (respectively, 42%) above the lower bound. The other algorithms yield significantly larger samples, with IncLing yielding the largest samples, with a median of more than twice the size of the lower bound. As not all feature models could be solved by all algorithms, the number of feature models solved by each algorithm is added in square brackets. Only the YASA variants and SampLNS could compute full samples for all feature models in time. IncLing and ICPL failed for one feature model, namely `financial_services`, to compute a feasible sample within the time limit. The other algorithms failed on multiple feature models.

6.2.2 (RQ2) Can SampLNS Compute Samples of Smaller Size Than Existing Algorithms. The mean sample sizes over five runs of SampLNS (with a 900 s time limit) are listed in the column *SampLNS UB* of Table 1. The values in parentheses are the smallest sample size found by SampLNS when running for up to 3 h, and only indicate the potential of SampLNS to further reduce the sample size. For 40 (85%) of the models, the *mean* sample size of SampLNS (after 900 s) was smaller than the *best* samples of any other algorithm. For 33 (70%) of the models, the mean sample size of SampLNS was at least 10% smaller than the smallest sample found by any other algorithm. At least 55% of the samples by SampLNS are provably optimal. On average, the samples of SampLNS are 22% smaller than the smallest sample found by any existing algorithm. The first bar in Figure 3 shows that the 75% quantile is with 10% smaller than the 25% quantile of YASA ($m = 10$) with 11%. To evaluate the variance of SampLNS, we computed the standard deviation of the relative differences from the best value, expressed as percentages. The mean standard deviation is 0.47% for the upper bound and 2.62% for the lower bound.

6.2.3 (RQ3) What Is the Computational Effort to Derive Lower Bounds and Optimize Samples with SampLNS. The average times achieved by SampLNS until the last improvement on the lower or upper bound are listed in column *Time to Bounds* in Table 1. At this time we could have aborted SampLNS while still achieving the same result. If SampLNS achieved a provably optimal sample size, it aborted at this time. For 21 of the feature models, this time is below 3 min. Figure 4 shows

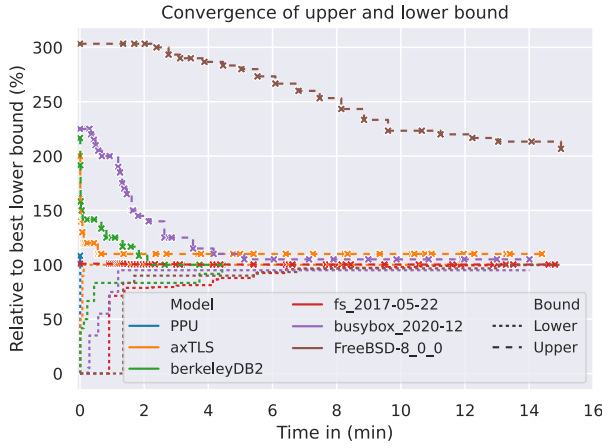


Fig. 4. Convergence of lower and upper bound over time for a selection of SamPLNS runs. Values are relative to the best lower bound of the model, so lower and upper bound meeting at 100% indicates provable optimality. Lower and upper bounds usually make quick progress already in the first seconds and minutes. The crosses on a curve indicate the end of an upper bound optimization step. The lower bound is only queried at the same time, such that not all increments are shown.

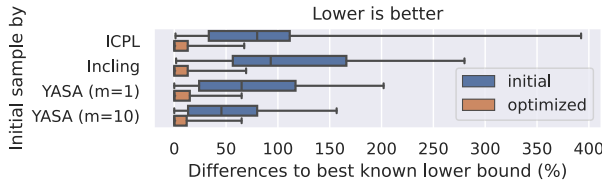


Fig. 5. Analogous to Figure 3, but comparing different versions of SamPLNS where the initial sample was produced by different algorithms, to compare the dependence of SamPLNS on the size of the initial sample. ICPL and Incling have significantly larger initial samples than YASA ($m = 10$) as can be seen by the blue boxes, but SamPLNS can still reduce them to a similar size as can be seen by the orange boxes. For YASA ($m = 1$) and YASA ($m = 10$), the optimized samples are nearly identical, despite a visible difference in the initial samples. This indicates that the size of the initial sample is of low importance for SamPLNS.

the convergence of lower and upper bounds over time for a selection of models. The curves for the remaining models are attached as artifact. *PPU* starts with an already optimal sample and terminates quickly after the matching lower bound has been found. The sample of *axTLS* gets quickly optimized but no matching lower bound can be found. For *berkeleyDB2*, we achieve strong improvements in the first seconds and then make continuous progress for 2 min. Optimality is proved in around 4 min. *fs_2017-05-22* already starts with a nearly optimal sample, which we only improve slightly and it takes around 7 min until we have a nearly tight lower bound. We also see multiple short optimization steps interleaved by a few longer steps. *busybox_2020-12-16* improves continuously over the first 5 min, first with many quick steps that become longer at the end. *FreeBSD-8_0_0* needs a few optimization steps before it makes any progress, but then makes continuous progress until the timeout.

6.2.4 (RQ4) Does SamPLNS Depend on the Sample Quality of YASA. Figure 5 shows the variance of the sample sizes of SamPLNS when exchanging the initial sample with samples generated by other algorithms. The runtime for computing the initial sample is not included in the time limit of

15 min, to allow for a fair comparison. Only the 46 feature models for which all algorithms could compute an initial sample in time are included. The sample sizes of the four selected algorithms have shown a high variance in Figure 3, but after optimization with SampLNS they all have a 75% quantile of 12% to 15% above the lower bound. The sample sizes of SampLNS are nearly identical when using YASA ($m = 1$) and YASA ($m = 10$), despite a visible difference in the initial samples. While the other two algorithms result in slightly larger samples, the difference is small when compared to the differences between the initial samples of the algorithms themselves.

6.3 Discussion

6.3.1 (RQ1) Can Existing Algorithms Compute Samples with a Sample Size Equal or Close to the Lower Bound. Yes, but only for few feature models in our benchmark set. For most feature models, none of the algorithms could compute a sample within 20% of neither the lower bound nor the smallest sample found using SampLNS. The best performing algorithm in our benchmark, YASA with $m = 10$, achieves for only 25% of the samples a gap of less than 13% (cf. Figure 3). As the effect of larger m for YASA is quickly diminishing, it also cannot be expected that fundamental improvements can be achieved by just increasing m . This is demonstrated by comparing YASA with $m = 10$, which takes an average of 15 s, to the variant that utilizes the full 900 s yet achieves only marginal improvements.

6.3.2 (RQ2) Can SampLNS Compute Samples of Smaller Size Than Existing Algorithms. Yes, and often substantially. SampLNS could minimize the samples for 85% of models in our benchmark set, reducing 55% of them to minimality and 82% below $1.2\times$ the lower bound. This not only highlights the performance of SampLNS, but also the quality of the lower bound for many feature models. It is worth mentioning that even small improvements on sample size are relevant for practical applications, as testing effort in terms of time and cost can be considerable for each tested configuration [33]. Especially in industrial product lines, consisting not only of software but also hardware components, for instance in the automotive domain, the effort and cost for assembling and testing even a single product can be enormous [39].

However, there are some feature models where the average sample size is still more than $1.7\times$ the lower bound. The sample size for the *FreeBDS-8_0_0* feature model was on average 47, whereas the best lower bound is only 30, leaving a large gap. Determining whether this gap stems from limitations of the lower bound or the sample generation is challenging, given the absence of knowledge regarding the true optimal sample size. The extended runtime experiments suggest there is room for enhancement on both fronts for models that have not been solved to optimality within 15 min.

The standard deviation of the lower bound, at 2.62%, is noticeably higher than that of the sample size, which is 0.47%. However, both values are relatively low, indicating that SampLNS consistently delivers reliable and good results.

6.3.3 (RQ3) What Is the Computational Effort to Derive Lower Bounds and Optimize Samples with SampLNS. SampLNS is efficient enough to optimize samples to (near-)optimality and compute lower bounds in seconds for small and in minutes for many medium-sized feature models. On most models, SampLNS makes visible improvements on the first few optimization steps, indicating a usefulness even for short optimization times. However, for some models the lower and upper bounds could still be visibly improved when running SampLNS for up to 3 h instead of only 900 s. The runtime of SampLNS is of course significantly higher than YASA ($m = 1$), but while increasing the optimization parameter m of YASA strongly increases the runtime and only leads to small improvements, SampLNS can achieve much larger improvements with a reasonable increase in runtime.

6.3.4 (RQ4) Does SampLNS Depend on the Sample Quality of YASA. No, the quality of the initial sample has only a negligible impact on SampLNS. Even when using the algorithm with the largest initial sample, SampLNS can still reduce the sample size to a similar size as when using the algorithm with the smallest initial sample. In particular, the increased optimization of YASA ($m = 10$) does not lead to a visible difference in the optimized sample size. This indicates that the quality of the initial sample is of low importance for SampLNS, and YASA could indeed be replaced by other algorithms without a significant impact on the results. As YASA ($m = 1$) is with an average of 2.1 s the fastest algorithm for generating an initial sample, it remains the best choice for SampLNS.

6.4 Threats to Validity

Internal Validity. With SampLNS, we reduce the sample size of a given sample, which brings the risk of lowering the t -wise coverage and introducing invalid configurations. To verify that the optimized samples are indeed correct (excluding errors in parser), we check the interactions of the initial samples for equality with the interactions of the optimized sample, and each configuration for feasibility. Finally, the coverage of all samples of all algorithms is checked for equality to also ensure correctness of the baseline algorithms.

In our evaluation, we use multiple sampling algorithms, as baseline and as input for SampLNS. Some algorithms are sensitive to the feature order of the input feature model, which leads to different samples. To mitigate the risk of this potential bias, we used a large corpus of different feature models and, in addition, repeated the sampling process for each feature model with each algorithm five times and selected the mean of the results. Furthermore, some sampling algorithms allow for additional parametrization. In particular, YASA allows to set its parameter m for the number of resamplings, which has an effect on the sample sizes. To mitigate this threat, we run YASA with different values for m , which are within a range that was also used in other measurements [57].

Repeated runs of SampLNS on the same feature model can lead to vastly different samples and certificates for the lower bound due to randomization. To mitigate this threat, we run SampLNS five times for each feature model and use the mean of the results. While the quality may differ, the randomization has no effect on the correctness of the results, as each optimization step preserves a valid t -wise sample.

We are comparing the sample sizes relative to the best lower bound found by SampLNS in Figures 3 to 5. However, the lower bound may not guarantee the absolute smallest sample size; it only ensures it is not larger. If we had a more precise lower bound, it could highlight differences more, showing our current approach tends to be cautious, possibly even disadvantaging SampLNS.

External Validity. In our evaluation, we used a wide range of different feature models (cf. Section 6.1). In this study, we took care to include feature models from many relevant domains, but did not include some large-scale systems, such as the Linux kernel. At this time, this leaves generalizing our results for feature models of other domains and with even higher numbers of features for followup studies; see Section 8 for an outlook.

We run SampLNS with specific parameter settings (cf. Section 5). Thus, we cannot generalize our results to other settings. However, as we did not focus on optimizing our initial settings, we can assume that an optimized parameter set (e.g., by using hyperparameter tuning) can yield even better results.

All sampling algorithms that we used in the evaluation are based on greedy strategies. While there are evolutionary algorithms that also guarantee a full t -wise coverage [31, 66], previous work suggests that such algorithms can be expected to produce results similar to the algorithms we considered [4].

7 Related Work

In the following, we state our work's relation to works on finding upper and lower bounds for t -wise samples, using other sample quality metrics and sampling strategies, applying general methods on test effort reduction for configurable systems, and computing covering arrays (i.e., samples) for unconstrained configurable systems.

Lower Bounds for t -Wise Samples. Yamada et al. [97] introduced CALOT for combinatorial testing, which uses an incremental SAT formulation that narrows down the sample size from an upper limit until it becomes infeasible, ultimately yielding a minimal sample. CALOT shares similarities with the CP-SAT formulation discussed in Section 4.1, but it employs a less powerful symmetry-breaking technique that would only yield weak lower bounds on binary features. Unlike our LNS approach, which partitions the solution space to maintain a manageable CP-SAT formulation size, CALOT's SAT formulation can become unwieldy. Another drawback is that its guarantee of optimality relies on the correctness of its implementation, and it cannot provide quality assurances if it runs out of time. While SAT-solvers can provide proofs for infeasibility, those proofs are significantly more complex than our certificates. Building on CALOT, Ansótegui et al. [8, 9] enhanced algorithm scalability, optimizing instances with even $t = 3, 4, 5$, but incomplete coverage. While using a refinement operator, the approaches are not as dynamic as our LNS. Furthermore, their lower bounding still just relies on a subset of t combinatorial features, unlike our comprehensive computation of maximal mutually excluding sets, potentially spanning all available features, which is essential for models only containing binary features.

Upper Bound for t -Wise Samples. There are numerous sampling algorithms for configurable systems that are designed for different use cases, based on different sampling strategies [3, 64, 92]. Most t -wise sampling algorithms, such as YASA [57], IncLing [4], UnWise [41], ScalableCA [65], ICPL [44], IPOG [60], Chvátal's algorithm [43], CASA [31], Alloy-based sampling [76], MoSo-PoLiTe [73], CALOT [97], and AETG-SAT [24], focus on finding small samples for full t -wise coverage. Consequently, any sample provided by one of these algorithms determines an upper bound for the used feature model; this is also the case for a recently proposed local search heuristic by Zhao et al. [99] on a different set of benchmarks. However, the distance from an optimal size of a t -wise sample (certified by a tight lower bound) was only known for very small instance sizes. This differs from our work, which focuses on computing *both*, upper *and* lower bounds that are close or equal to the optimal size of a pairwise sample.

Sample Quality Metrics. SampLNS tries to find optimal t -wise samples in terms of sample size. However, the quality of sampling algorithms can also be assessed by other criteria [92], including sampling efficiency [4, 31, 35, 44, 57, 66, 74, 78], effectiveness [4, 33, 68, 87], and stability [77]. Regarding sampling efficiency, SampLNS allows to specify a time limit for optimizing a sample, similar to many evolutionary algorithms [22, 27, 30, 36–38]. Determining the effectiveness of a sample is dependent on the particular use case, such as fault detection potential for testing [33, 68]. However, as this is hard to evaluate, many other coverage criteria are used as a proxy instead [92].

Beyond t -Wise Sampling. SampLNS considers t -wise interaction coverage, but there exist many other coverage criteria for which samples can be generated [92], such as partial t -wise coverage [15, 45, 54, 68], coverage of the solution space [39, 56, 86, 89], coverage of test cases [13, 50, 51], uniform sampling [2, 40, 72, 81, 84], distance-based sampling [37, 46], and coverage of feature model mutations [12, 36, 83]. As some of these coverage criteria are correlated to t -wise coverage, it is reasonable to assume that LNS may be employed to reduce the size of corresponding samples as well.

Test Effort Reduction for Configurable Systems. The ultimate goal of reducing sample sizes, in context of testing configurable systems, is to reduce the test effort. Besides sample size reduction, this goal may also be achieved by, for instance, test case selection [13, 14], test case prioritization [5, 7, 61], and regression testing [62, 63, 71, 98]. Such approaches are orthogonal to our technique and may be used complementary to SampLNS.

Analysis Strategies for Configurable Systems. Using sample-based testing, such as t -wise sampling, is just one strategy for analyzing interactions in configurable systems. Alternative approaches encompass family-based [71, 90, 94, 95] and regression-based analysis [62, 63, 71, 98]. In contrast to sample-based testing, which can rely on single-system tooling, family- and regression-based analyses require special tooling to work [90]. In addition, family-based testing often does not scale for large-scale systems [48, 70].

Covering Arrays for Unconstrained Configurable Systems. Finding lower and upper bounds for unconstrained combinatorial interaction testing has received intensive attention [20, 23, 34, 49, 52]. For unconstrained configuration spaces, Cohen et al. [23] show that the minimal size for a t -wise sample grows logarithmically in the number of features. Furthermore, for unconstrained Boolean feature models, optimal pairwise samples are known for any number of features [34, 49, 52] and reasonable lower and upper bounds are known for t -wise samples [20, 34]. We consider constrained Boolean feature models, which makes finding an optimal t -wise sample and even lower and upper bounds substantially more difficult [96].

8 Conclusion

Quality assurance of modern software systems is often challenged by their large configuration spaces. The sample size is of crucial importance, as it is generally considered the dominating factor for the test effort for configurable systems [24, 34, 49, 52]. In the last two decades, many heuristics have been proposed to derive sample configurations, but the quality of their results could only be compared to other heuristics so far. In particular, it has been unclear *how low we can go* when minimizing sample size of t -wise interaction samples, i.e., how close heuristically achieved solutions actually are to being optimal.

With SampLNS, we propose a new algorithm that offers a significant advancement in minimizing the size of t -wise interaction samples to near-optimality. Importantly, it also provides easily verifiable lower bounds to prove the solution quality. Our algorithm relies on CP and MIP, which are scaled to the complexity of configuration spaces by means of a LNS. Our empirical evaluation shows that SampLNS scales to configuration spaces with at least up to 1,400 configuration options. For 26 of 47 subject systems, sample sizes of SampLNS match the lower bounds, so we can provide certificates that prove their optimality. In contrast, the samples computed by any previous algorithm are optimal for only seven systems. Even if samples generated with SampLNS are not guaranteed to be optimal for 21 systems, sample size and lower bounds are often close (e.g., off by only one for five systems). In addition, SampLNS was able to improve the sample size for 40 of 47 systems compared to the previous best algorithm for each system.

Our algorithm significantly reduces the testing effort for small- and medium-sized configurable systems. Additionally, our lower bounds offer a clear framework for potential improvements, enabling the differentiation between instances with a small remaining optimality gap (and thus minimal room for improvement) and those with a large gap (and consequently greater potential for improvement). These insights open up various directions for future work.

One promising avenue is extending the range of instance sizes that can be solved to provable optimality; currently, the largest such instance is EMBToolkit with 1,179 features. Achieving this goal may be possible through a combination of hyperparameter tuning and more efficient data

structures. While we do not expect this approach to yield provably optimal solutions for very large instances of practical importance—such as those arising from the Linux kernel, which features nearly 20,000 options—we remain optimistic that a spectrum of Algorithm Engineering methods could provide substantial improvements. These may include further refinements to achieve better upper bounds and mathematical enhancements to the dual problem, aimed at generating provable lower bounds when the current dual leaves a nontrivial duality gap.

Moreover, we are hopeful that the fundamental ideas demonstrated in this work can be applied to other computationally complex problems in configurable systems beyond minimizing the sample size in t -wise interaction sampling.

References

- [1] Iago Abal, Jean Melo, Stefan Stănculescu, Claus Brabrand, Márcio Ribeiro, and Andrzej Wasowski. 2018. Variability bugs in highly configurable systems: A qualitative analysis. *Transactions on Software Engineering and Methodology* 26, 3 (Jan. 2018), 10:1–10:34. DOI: <https://doi.org/10.1145/3149119>
- [2] Dimitris Achlioptas, Zayd S. Hammoudeh, and Panos Theodoropoulos. 2018. Fast sampling of perfectly uniform satisfying assignments. In *Proceedings of the International Conference on Theory and Applications of Satisfiability Testing (SAT)*. Springer, Berlin, Germany, 135–147.
- [3] Bestoun S. Ahmed, Kamal Z. Zamli, Wasif Afzal, and Miroslav Bures. 2017. Constrained interaction testing: A systematic literature study. *IEEE Access* 5 (2017), 25706–25730. DOI: <https://doi.org/10.1109/ACCESS.2017.2771562>
- [4] Mustafa Al-Hajjaji, Sebastian Krieter, Thomas Thüm, Malte Lochau, and Gunter Saake. 2016. IncLing: Efficient product-line testing using incremental pairwise sampling. In *Proceedings of the International Conference on Generative Programming: Concepts & Experiences (GPCE)*. ACM, New York, NY, 144–155. DOI: <https://doi.org/10.1145/2993236.2993253>
- [5] Mustafa Al-Hajjaji, Sascha Lity, Remo Lachmann, Thomas Thüm, Ina Schaefer, and Gunter Saake. 2017. Delta-oriented product prioritization for similarity-based product-line testing. In *Proceedings of the International Workshop on Variability and Complexity in Software Design (VACE)*. IEEE, Piscataway, NJ, 34–40. DOI: <https://doi.org/10.1109/VACE.2017.8>
- [6] Mustafa Al-Hajjaji, Jens Meinicke, Sebastian Krieter, Reimar Schröter, Thomas Thüm, Thomas Leich, and Gunter Saake. 2016. Tool demo: Testing configurable systems with FeatureIDE. In *Proceedings of the International Conference on Generative Programming: Concepts & Experiences (GPCE)*. ACM, New York, NY, 173–177. DOI: <https://doi.org/10.1145/2993236.2993254>
- [7] Mustafa Al-Hajjaji, Thomas Thüm, Malte Lochau, Jens Meinicke, and Gunter Saake. 2019. Effective product-line testing using similarity-based product prioritization. *Software and System Modeling* 18, 1 (Feb. 2019), 499–521. DOI: <https://doi.org/10.1007/s10270-016-0569-2>
- [8] Carlos Ansótegui, Felip Manyà, Jesus Ojeda, Josep M. Salvia, and Eduard Torres. 2022. Incomplete MaxSAT approaches for combinatorial testing. *Journal of Heuristics* 28, 4 (2022), 377–431.
- [9] Carlos Ansótegui, Jesús Ojeda, and Eduard Torres. 2021. Building high strength mixed covering arrays with constraints. In *Proceedings of the 27th International Conference on Principles and Practice of Constraint Programming (CP '21)*. Laurent D. Michel (Ed.), Vol. 210, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 12:1–12:17. DOI: <https://doi.org/10.4230/LIPIcs.CP.2021.12>
- [10] Sven Apel, Don Batory, Christian Kästner, and Gunter Saake. 2013. *Feature-Oriented Software Product Lines*. Springer, Berlin, Germany. DOI: <https://doi.org/10.1007/978-3-642-37521-7>
- [11] David Applegate, Robert Bixby, William Cook, and Vasek Chvátal. 1998. On the solution of traveling salesman problems. *Documenta Mathematica* (1998), 645–656.
- [12] Paolo Arcaini, Angelo Gargantini, and Paolo Vavassori. 2015. Generating tests for detecting faults in feature models. In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 1–10. DOI: <https://doi.org/10.1109/ICST.2015.7102591>
- [13] Hauke Baller, Sascha Lity, Malte Lochau, and Ina Schaefer. 2014. Multi-objective test suite optimization for incremental product family testing. In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Washington, DC, 303–312. DOI: <https://doi.org/10.1109/ICST.2014.43>
- [14] Hauke Baller and Malte Lochau. 2014. Towards incremental test suite optimization for software product lines. In *Proceedings of the International Workshop on Feature-Oriented Software Development (FOSD)*. ACM, New York, NY, 30–36. DOI: <https://doi.org/10.1145/2660190.2662115>

- [15] Eduard Baranov, Axel Legay, and Kuldeep S. Meel. 2020. Baital: An adaptive weighted sampling approach for improved t-wise coverage. In *Proceedings of the European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, 1114–1126. DOI: <https://doi.org/10.1145/3368089.3409744>
- [16] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximilian Heisinger. 2020. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020. In *Proceedings of the SAT Competition 2020 – Solver and Benchmark Descriptions*. Tomas Balyo, Nils Froleyks, Marijn Heule, Markus Iser, Matti Järvisalo, and Martin Suda (Eds.), Vol. B-2020-1, University of Helsinki, 51–53.
- [17] Thomas F. Bowen, Frank S. Dworack, Ching-Hua Chow, Nancy Griffeth, Gary E. Herman, and Yow-Jian Lin. 1989. The feature interaction problem in telecommunications systems. In *Proceedings of the International Conference on Software Engineering for Telecommunication Switching Systems (SETSS)*. IEEE, Washington, DC, 59–62.
- [18] Muffy Calder, Mario Kolberg, Evan H. Magill, and Stephan Reiff-Marganiec. 2003. Feature interaction: A critical review and considered forecast. *Computer Networks* 41, 1 (2003), 115–141.
- [19] Ivan Do Carmo Machado, John D. McGregor, Yguarã Cerqueira Cavalcanti, and Eduardo Santana De Almeida. 2014. On strategies for testing software product lines: A systematic literature review. *Journal of Information and Software Technology* 56, 10 (2014), 1183–1199. DOI: <https://doi.org/10.1016/j.infsof.2014.04.002>
- [20] Soohak Choi, Hyun Kwang Kim, and Dong Yeol Oh. 2012. Structures and lower bounds for binary covering arrays. *Discrete Mathematics* 312, 19 (2012), 2958–2968. DOI: <https://doi.org/10.1016/j.disc.2012.06.013>
- [21] Vasek Chvatal. 1979. A greedy heuristic for the set-covering problem. *Mathematics of Operations Research* 4, 3 (1979), 233–235.
- [22] Anastasia Cmyrev and Ralf Reissing. 2014. Efficient and effective testing of automotive software product lines. *International Journal of Applied Science and Technology* 7, 2 (2014), 53–57. DOI: <https://doi.org/10.14416/j.ijast.2014.05.001>
- [23] David M. Cohen, Siddhartha R. Dalal, Michael L. Fredman, and Gardner C. Patton. 1997. The AETG system: An approach to testing based on combinatorial design. *IEEE Transactions on Software Engineering* 23, 7 (1997), 437–444. DOI: <https://doi.org/10.1109/32.605761>
- [24] Myra B. Cohen, Matthew B. Dwyer, and Jiangfan Shi. 2008. Constructing interaction test suites for highly-configurable systems in the presence of constraints: A greedy approach. *IEEE Transactions on Software Engineering* 34, 5 (2008), 633–650.
- [25] William J. Cook, David L. Applegate, Robert E. Bixby, and Vašek Chvátal. 2011. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press.
- [26] Emilie Danna, Edward Rothberg, and Claude Le Pape. 2005. Exploring relaxation induced neighborhoods to improve MIP solutions. *Mathematical Programming* 102 (2005), 71–90.
- [27] Faezeh Ensan, Ebrahim Bagheri, and Dragan Gasevic. 2012. Evolutionary search-based test generation for software product line feature models. In *Proceedings of the International Conference on Advanced Information Systems Engineering (CAiSE)*, Vol. 7328. Springer, 613–628. DOI: https://doi.org/10.1007/978-3-642-31095-9_40
- [28] Fischer Ferreira, Gustavo Vale, João Paulo Diniz, and Eduardo Figueiredo. 2021. Evaluating t-wise testing strategies in a community-wide dataset of configurable software systems. *Journal of Systems and Software* 179 (2021), 110990. DOI: <https://doi.org/10.1016/j.jss.2021.110990>
- [29] Thibaut Feydy and Peter J. Stuckey. 2009. Lazy clause generation reengineered. In *Proceedings of the 15th International Conference on Principles and Practice of Constraint Programming (CP '09)*. Springer, 352–366.
- [30] Helson L. Jakubowski Filho, Jackson A. Prado Lima, and Silvia R. Vergilio. 2017. Automatic generation of search-based algorithms applied to the feature testing of software product lines. In *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*. ACM, New York, NY, 114–123. DOI: <https://doi.org/10.1145/3131151.3131152>
- [31] Brady J. Garvin, Myra B. Cohen, and Matthew B. Dwyer. 2011. Evaluating improvements to a meta-heuristic search for constrained interaction testing. *Empirical Software Engineering* 16, 1 (2011), 61–102. DOI: <https://doi.org/10.1007/s10664-010-9135-7>
- [32] Gurobi Optimization, LLC. 2023. Gurobi Optimizer Reference Manual. Retrieved from <https://www.gurobi.com>
- [33] Axel Halin, Alexandre Nuttinck, Mathieu Acher, Xavier Devroey, Gilles Perrouin, and Benoit Baudry. 2019. Test them all, is it worth it? Assessing configuration sampling on the JHipster web development stack. *Empirical Software Engineering* 24, 2 (July 2019), 674–717.
- [34] Alan Hartman. 2006. Software and hardware testing using combinatorial covering suites. In *Graph Theory, Combinatorics and Algorithms: Interdisciplinary Applications*. Martin Charles Golumbic and Irith Ben-Arroyo Hartman (Eds.), Vol. 34, Springer, 237–266.
- [35] Evelyn Nicole Haslinger, Roberto E. Lopez-Herrejon, and Alexander Egyed. 2013. Using feature model knowledge to speed up the generation of covering arrays. In *Proceedings of the International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, 16:1–16:6. DOI: <https://doi.org/10.1145/2430502.2430524>

- [36] Christopher Henard, Mike Papadakis, and Yves Le Traon. 2014. Mutation-based generation of software product line test configurations. In *Search-Based Software Engineering*. Claire Le Goues and Shin Yoo (Eds.), Lecture Notes in Computer Science, Vol. 8636, Springer International Publishing, 92–106. DOI: https://doi.org/10.1007/978-3-319-09940-8_7
- [37] Christopher Henard, Mike Papadakis, Gilles Perrouin, Jacques Klein, Patrick Heymans, and Yves Le Traon. 2014. Bypassing the combinatorial explosion: Using similarity to generate and prioritize t-wise test configurations for software product lines. *IEEE Transactions on Software Engineering* 40, 7 (July 2014), 650–670. DOI: <https://doi.org/10.1109/TSE.2014.2327020>
- [38] Christopher Henard, Mike Papadakis, Gilles Perrouin, Jacques Klein, and Yves Le Traon. 2013. Multi-objective test generation for software product lines. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 62–71. DOI: <https://doi.org/10.1145/2491627.2491635>
- [39] Marc Hentze, Tobias Pett, Chico Sundermann, Sebastian Krieter, Thomas Thüm, and Ina Schaefer. 2022. Generic solution-space sampling for multi-domain product lines. In *Proceedings of the International Conference on Generative Programming: Concepts & Experiences (GPCE)*. ACM, New York, NY, 135–147.
- [40] Ruben Heradio, David Fernández-Amorós, José Antonio Galindo, and David Benavides. 2020. Uniform and scalable SAT-sampling for configurable systems. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 17:1–17:11.
- [41] Tobias Heß, Tim Jannik Schmidt, Lukas Ostheimer, Sebastian Krieter, and Thomas Thüm. 2024. UnWise: High t-wise coverage from uniform sampling. In *Proceedings of the International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS '24)*. ACM, New York, NY, 37–45. DOI: <https://doi.org/10.1145/3634713.3634716>
- [42] Claus Hunsen, Bo Zhang, Janet Siegmund, Christian Kästner, Olaf Leßenich, Martin Becker, and Sven Apel. 2016. Preprocessor-based variability in open-source and industrial software systems: An empirical study. *Empirical Software Engineering* 21, 2 (April 2016), 449–482. DOI: <https://doi.org/10.1007/s10664-015-9360-1>
- [43] Martin Fagereng Johansen, Øystein Haugen, and Franck Fleurey. 2011. Properties of realistic feature models make combinatorial testing of product lines feasible. In *Proceedings of the International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Springer, Berlin, Germany, 638–652. DOI: https://doi.org/10.1007/978-3-642-24485-8_47
- [44] Martin Fagereng Johansen, Øystein Haugen, and Franck Fleurey. 2012. An algorithm for generating t-wise covering arrays from large feature models. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 46–55. DOI: <https://doi.org/10.1145/2362536.2362547>
- [45] Martin Fagereng Johansen, Øystein Haugen, Franck Fleurey, Anne Grete Eldegard, and Torbjørn Syversen. 2012. Generating better partial covering arrays by modeling weights on sub-product lines. In *Proceedings of the International Conference on Model Driven Engineering Languages and Systems (MODELS)*. Springer, Berlin, Germany, 269–284. DOI: https://doi.org/10.1007/978-3-642-33666-9_18
- [46] Christian Kaltenecker, Alexander Grebhahn, Norbert Siegmund, Jianmei Guo, and Sven Apel. 2019. Distance-based sampling of software configuration spaces. In *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE, Piscataway, NJ, 1084–1094. DOI: <https://doi.org/10.1109/ICSE.2019.00112>
- [47] Richard M. Karp. 1972. Reducibility among combinatorial problems. In *Proceedings of a Symposium on the Complexity of Computer Computations*. Springer, 85–103.
- [48] Christian Kästner, Alexander von Rhein, Sebastian Erdweg, Jonas Pusch, Sven Apel, Tillmann Rendel, and Klaus Ostermann. 2012. Toward variability-aware testing. In *Proceedings of the International Workshop on Feature-Oriented Software Development (FOSD)*. ACM, New York, NY, 1–8. DOI: <https://doi.org/10.1145/2377816.2377817>
- [49] G. O. H. Katona. 1973. Two applications (for search theory and truth functions) of Sperner type theorems. *Periodica Mathematica Hungarica* 3, 1 (1973), 19–26. DOI: <https://doi.org/10.1007/BF02018457>
- [50] Chang Hwan Peter Kim, Don Batory, and Sarfraz Khurshid. 2011. Reducing combinatorics in testing product lines. In *Proceedings of the International Conference on Aspect-Oriented Software Development (AOSD)*. ACM, New York, NY, 57–68. DOI: <https://doi.org/10.1145/1960275.1960284>
- [51] Chang Hwan Peter Kim, Darko Marinov, Sarfraz Khurshid, Don Batory, Sabrina Souto, Paulo Barros, and Marcelo D’Amorim. 2013. SPLat: Lightweight dynamic analysis for reducing combinatorics in testing configurable systems. In *Proceedings of the European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, 257–267. DOI: <https://doi.org/10.1145/2491411.2491459>
- [52] Daniel J. Kleitman and Joel Spencer. 1973. Families of k-independent sets. *Discrete Mathematics* 6, 3 (1973), 255–262. DOI: [https://doi.org/10.1016/0012-365X\(73\)90098-8](https://doi.org/10.1016/0012-365X(73)90098-8)
- [53] Alexander Knüppel, Thomas Thüm, Stephan Mennicke, Jens Meinicke, and Ina Schaefer. 2017. Is there a mismatch between real-world feature models and product-line research? In *Proceedings of the European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE)*. ACM, New York, NY, 291–302. DOI: <https://doi.org/10.1145/3106237.3106252>

- [54] Matthias Kowal, Sandro Schulze, and Ina Schaefer. 2013. Towards efficient SPL testing by variant reduction. In *Proceedings of the International Workshop on Variability and Composition (VariComp)*. ACM, New York, NY, 1–6. DOI : <https://doi.org/10.1145/2451617.2451619>
- [55] Sebastian Krieter, Marcus Pinnecke, Jacob Krüger, Joshua Sprey, Christopher Sontag, Thomas Thüm, Thomas Leich, and Gunter Saake. 2017. FeatureIDE: Empowering third-party developers. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 42–45. DOI : <https://doi.org/10.1145/3109729.3109751>
- [56] Sebastian Krieter, Thomas Thüm, Sandro Schulze, Sebastian Ruland, Malte Lochau, Gunter Saake, and Thomas Leich. 2022. *T-Wise Presence Condition Coverage and Sampling for Configurable Systems*. Technical Report arXiv:2205.15180. Cornell University Library. DOI : <https://doi.org/10.48550/arXiv.2205.15180>
- [57] Sebastian Krieter, Thomas Thüm, Sandro Schulze, Gunter Saake, and Thomas Leich. 2020. YASA: Yet another sampling algorithm. In *Proceedings of the International Working Conference on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, Article 4, 10 pages. DOI : <https://doi.org/10.1145/3377024.3377042>
- [58] Elias Kuitert, Sebastian Krieter, Chico Sundermann, Thomas Thüm, and Gunter Saake. 2022. Tseitin or not tseitin? The impact of CNF transformations on feature-model analyses. In *Proceedings of the International Conference on Automated Software Engineering (ASE)*. ACM, New York, NY, 110:1–110:13.
- [59] Y. C. Law and J. H. M. Lee. 2006. Symmetry breaking constraints for value symmetries in constraint satisfaction. *Constraints* 11 (2006), 221–267.
- [60] Yu Lei, Raghu N. Kacker, D. Richard Kuhn, Vadim Okun, and James Lawrence. 2007. IPOG: A general strategy for t-way software testing. In *Proceedings of the International Conference on Engineering of Computer-Based Systems (ECBS)*. IEEE, 549–556.
- [61] Sascha Lity, Mustafa Al-Hajjaji, Thomas Thüm, and Ina Schaefer. 2017. Optimizing product orders using graph algorithms for improving incremental product-line analysis. In *Proceedings of the International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, 60–67. DOI : <https://doi.org/10.1145/3023956.3023961>
- [62] Sascha Lity, Thomas Morbach, Thomas Thüm, and Ina Schaefer. 2016. Applying incremental model slicing to product-line regression testing. In *Proceedings of the International Conference on Software Reuse (ICSR)*. Springer, Berlin, Germany, 3–19. DOI : https://doi.org/10.1007/978-3-319-35122-3_1
- [63] Sascha Lity, Manuel Nieke, Thomas Thüm, and Ina Schaefer. 2019. Retest test selection for product-line regression testing of variants and versions of variants. *Journal of Systems and Software* 147 (Jan. 2019), 46–63. DOI : <https://doi.org/10.1016/j.jss.2018.09.090>
- [64] Roberto E. Lopez-Herrejon, Stefan Fischer, Rudolf Ramler, and Aalexander Egyed. 2015. A first systematic mapping study on combinatorial interaction testing for software product lines. In *Proceedings of the International Workshop on Combinatorial Testing (IWCT)*. IEEE, Washington, DC, 1–10. DOI : <https://doi.org/10.1109/ICSTW.2015.7107435>
- [65] Chuan Luo, Shuangyu Lyu, Qiyuan Zhao, Wei Wu, Hongyu Zhang, and Chunming Hu. 2024. Beyond pairwise testing: Advancing 3-wise combinatorial interaction testing for highly configurable systems. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 641–653.
- [66] Dusica Marijan, Arnaud Gotlieb, Sagar Sen, and Aymeric Hervieu. 2013. Practical pairwise testing for software product lines. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 227–235. DOI : <https://doi.org/10.1145/2491627.2491646>
- [67] Joao P. Marques-Silva and Karem A. Sakallah. 1999. GRASP: A search algorithm for propositional satisfiability. *IEEE Transactions on Computers* 48, 5 (1999), 506–521.
- [68] Flávio Medeiros, Christian Kästner, Márcio Ribeiro, Rohit Gheyi, and Sven Apel. 2016. A comparison of 10 sampling algorithms for configurable systems. In *Proceedings of the International Conference on Software Engineering (ICSE)*. ACM, New York, NY, 643–654. DOI : <https://doi.org/10.1145/2884781.2884793>
- [69] Jens Meinicke, Thomas Thüm, Reimar Schröter, Fabian Benduhn, Thomas Leich, and Gunter Saake. 2017. *Mastering Software Variability with FeatureIDE*. Springer, Berlin, Germany. DOI : <https://doi.org/10.1007/978-3-319-61443-4>
- [70] Jens Meinicke, Chu-Pan Wong, Christian Kästner, Thomas Thüm, and Gunter Saake. 2016. On essential configuration complexity: Measuring interactions in highly-configurable systems. In *Proceedings of the International Conference on Automated Software Engineering (ASE)*. ACM, New York, NY, 483–494. DOI : <https://doi.org/10.1145/2970276.2970322>
- [71] Yannic Noller, Hoang Lam Nguyen, Minxing Tang, Timo Kehrer, and Lars Grunske. 2021. Complete shadow symbolic execution with Java PathFinder. *SIGSOFT Software Engineering Notes* 44, 4 (Sept. 2021), 15–16. DOI : <https://doi.org/10.1145/3364452.33644558>
- [72] Jeho Oh, Paul Gazzillo, and Don Batory. 2019. t-Wise coverage by uniform sampling. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, 84–87.
- [73] Sebastian Oster, Florian Markert, and Philipp Ritter. 2010. Automated incremental pairwise testing of software product lines. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. Springer, Berlin, Germany, 196–210. DOI : https://doi.org/10.1007/978-3-642-15579-6_14

- [74] Sebastian Oster, Ivan Zorcic, Florian Markert, and Malte Lochau. 2011. MoSo-PoLiTe - Tool support for pairwise and model-based software product line testing. In *Proceedings of the International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, 79–82. DOI : <https://doi.org/10.1145/1944892.1944901>
- [75] Laurent Perron and Vincent Furnon. 2024. OR-Tools. Retrieved from <https://developers.google.com/optimization/>
- [76] Gilles Perrouin, Sagar Sen, Jacques Klein, Benoit Baudry, and Yves Le Traon. 2010. Automated and scalable t-wise test case generation strategies for software product lines. In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Washington, DC, 459–468. DOI : <https://doi.org/10.1109/ICST.2010.43>
- [77] Tobias Pett, Sebastian Krieter, Tobias Runge, Thomas Thüm, Malte Lochau, and Ina Schaefer. 2021. Stability of product-line sampling in continuous integration. In *Proceedings of the International Working Conference on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, Article 18, 9 pages. DOI : <https://doi.org/10.1145/3442391.3442410>
- [78] Tobias Pett, Sebastian Krieter, Thomas Thüm, and Ina Schaefer. 2024. MulTi-wise sampling: Trading uniform t-wise feature interaction coverage for smaller samples. In *Proceedings of the International Systems and Software Product Line Conference (SPLC '24)*. ACM, New York, NY, 47–53. DOI : <https://doi.org/10.1145/3646548.3672589>
- [79] Tobias Pett, Thomas Thüm, Tobias Runge, Sebastian Krieter, Malte Lochau, and Ina Schaefer. 2019. Product sampling for product lines: The scalability challenge. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 78–83. DOI : <https://doi.org/10.1145/3336294.3336322>
- [80] David Pisinger and Stefan Ropke. 2019. Large neighborhood search. In *Handbook of Metaheuristics*. Gendreau Michel and Potvin Jean-Yves (Eds.), Springer, 99–127.
- [81] Quentin Plazar, Mathieu Acher, Gilles Perrouin, Xavier Devroey, and Maxime Cordy. 2019. Uniform sampling of SAT solutions for configurable systems: Are we there yet? In *Proceedings of the International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, Piscataway, NJ, 240–251. DOI : <https://doi.org/10.1109/ICST.2019.00032>
- [82] Rodrigo Queiroz, Leonardo Passos, Marco Tulio Valente, Claus Hunsen, Sven Apel, and Krzysztof Czarnecki. 2017. The shape of feature code: An analysis of twenty C-preprocessor-based systems. *Software and System Modeling* 16, 1 (Feb. 2017), 77–96. DOI : <https://doi.org/10.1007/s10270-015-0483-z>
- [83] Dennis Reuling, Johannes Bürdek, Serge Rotärmel, Malte Lochau, and Udo Kelter. 2015. Fault-based product-line testing: Effective sample generation based on feature-diagram mutation. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 131–140. DOI : <https://doi.org/10.1145/2791060.2791074>
- [84] Shubham Sharma, Rahul Gupta, Subhajit Roy, and Kuldeep S. Meel. 2018. Knowledge compilation meets uniform sampling. In *Proceedings of the International Conference on Logic for Programming, Artificial Intelligence, and Reasoning*. EasyChair, 620–636.
- [85] Steven She, Rafael Lotufo, Thorsten Berger, Andrzej Wąsowski, and Krzysztof Czarnecki. 2011. Reverse engineering feature models. In *Proceedings of the International Conference on Software Engineering (ICSE)*. ACM, New York, NY, 461–470. DOI : <https://doi.org/10.1145/1985793.1985856>
- [86] Jiangfan Shi, Myra B. Cohen, and Matthew B. Dwyer. 2012. Integration testing of software product lines using compositional symbolic execution. In *Proceedings of the International Conference on Fundamental Approaches to Software Engineering (FASE)*. Springer, Berlin, Germany, 270–284. DOI : https://doi.org/10.1007/978-3-642-28872-2_19
- [87] Norbert Siegmund, Marko Rosenmüller, Martin Kuhlemann, Christian Kästner, Sven Apel, and Gunter Saake. 2012. SPL conqueror: Toward optimization of non-functional properties in software product lines. *Software Quality Journal* 20, 3–4 (Sept. 2012), 487–517. DOI : <https://doi.org/10.1007/s11219-011-9152-9>
- [88] Chico Sundermann, Tobias Heß, Michael Nieke, Paul Maximilian Bittner, Jeffrey M. Young, Thomas Thüm, and Ina Schaefer. 2023. Evaluating state-of-the-art SAT solvers on industrial configuration spaces. *Empirical Software Engineering* 28, 2 (2023), 29.
- [89] Reinhard Tartler, Daniel Lohmann, Christian Dietrich, Christoph Egger, and Julio Sincero. 2012. Configuration coverage in the analysis of large-scale system software. *ACM SIGOPS Operating Systems Review* 45, 3 (Jan. 2012), 10–14. DOI : <https://doi.org/10.1145/2039239.2039242>
- [90] Thomas Thüm, Sven Apel, Christian Kästner, Ina Schaefer, and Gunter Saake. 2014. A classification and survey of analysis strategies for software product lines. *ACM Computing Surveys* 47, 1 (Jun. 2014), 6:1–6:45. DOI : <https://doi.org/10.1145/2580950>
- [91] Thomas Thüm, Christian Kästner, Sebastian Erdweg, and Norbert Siegmund. 2011. Abstract features in feature model modeling. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. IEEE, Washington, DC, 191–200. DOI : <https://doi.org/10.1109/SPLC.2011.53>
- [92] Mahsa Varshosaz, Mustafa Al-Hajjaji, Thomas Thüm, Tobias Runge, Mohammad Reza Mousavi, and Ina Schaefer. 2018. A classification of product sampling for software product lines. In *Proceedings of the International Systems and Software Product Line Conference (SPLC)*. ACM, New York, NY, 1–13. DOI : <https://doi.org/10.1145/3233027.3233035>

- [93] Alexander von Rhein, Sven Apel, Christian Kästner, Thomas Thüm, and Ina Schaefer. 2013. The PLA model: On the combination of product-line analyses. In *Proceedings of the International Workshop on Variability Modelling of Software-Intensive Systems (VaMoS)*. ACM, New York, NY, 14:1–14:8. DOI : <https://doi.org/10.1145/2430502.2430515>
- [94] Chu-Pan Wong, Jens Meinicke, and Christian Kästner. 2018. Beyond testing configurable systems: Applying variational execution to automatic program repair and higher order mutation testing. In *Proceedings of the European Software Engineering Conference/Foundations of Software Engineering (ESEC/FSE '18)*. ACM, New York, NY, 749–753. DOI : <https://doi.org/10.1145/3236024.3264837>
- [95] Chu-Pan Wong, Jens Meinicke, Lukas Lazarek, and Christian Kästner. 2018. Faster variational execution with transparent bytecode transformation. *PACM on Programming Languages* 2, OOPSLA, Article 117 (Oct. 2018), 30 pages. DOI : <https://doi.org/10.1145/3276487>
- [96] Huayao Wu, Changhai Nie, Justyna Petke, Yue Jia, and Mark Harman. 2019. A survey of constrained combinatorial testing. arXiv:1908.02480. Retrieved from <http://arxiv.org/abs/1908.02480>
- [97] Akihisa Yamada, Takashi Kitamura, Cyrille Artho, Eun-Hye Choi, Yutaka Oiwa, and Armin Biere. 2015. Optimization of combinatorial testing by incremental SAT solving. In *Proceedings of the 2015 IEEE 8th International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 1–10.
- [98] Shin Yoo and Mark Harman. 2012. Regression testing minimization, selection and prioritization: A survey. *Software Testing, Verification and Reliability* 22, 2 (2012), 67–120.
- [99] Qiyuan Zhao, Chuan Luo, Shaowei Cai, Wei Wu, Jinkun Lin, Hongyu Zhang, and Chunming Hu. 2023. CAmptactor: A novel and effective local search algorithm for optimizing pairwise covering arrays. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 81–93.

Received 3 May 2024; revised 9 December 2024; accepted 11 December 2024